

# HookSuite

Herramienta de auditoría de seguridad web con Inteligencia Artificial

Módulo · Ciberseguridad Avanzada | Curso 2026

Práctica · Práctica 1 — Red Team

## EQUIPO DE DESARROLLO

|    |                  |                                     |
|----|------------------|-------------------------------------|
| P1 | Ivan Medina      | Frontend — React + Vite + Tailwind  |
| P2 | Macarena Rogerio | Backend — FastAPI + Python          |
| P3 | Nacho García     | Playwright — Automatización         |
| P4 | Carlos Bañuelos  | DevTools — Chrome DevTools Protocol |
| P5 | Jose María López | IA + GitHub — Claude API            |

# Índice de contenidos

---

|                                                                  |    |
|------------------------------------------------------------------|----|
| 1. Resumen ejecutivo .....                                       | 4  |
| 2. Análisis y comparación con otras herramientas .....           | 5  |
| 2.1. El ecosistema actual de herramientas de auditoría web ..... | 5  |
| 2.2. La propuesta de HookSuite .....                             | 5  |
| 2.3. Relevancia en el contexto académico .....                   | 6  |
| 3. Arquitectura técnica .....                                    | 7  |
| 3.1. Visión general del sistema .....                            | 7  |
| 3.2. Proceso de decisión arquitectónica .....                    | 7  |
| 3.3. Stack tecnológico .....                                     | 8  |
| 3.4. Contratos de integración activos — Práctica 1 .....         | 9  |
| 4. Proceso de desarrollo .....                                   | 10 |
| 4.1. Proceso individual .....                                    | 10 |
| 4.2. Integración .....                                           | 26 |
| 4.3. Estado actual de la herramienta .....                       | 39 |
| 5. Guía de despliegue .....                                      | 50 |
| 5.1. Entorno de producción .....                                 | 50 |
| 5.2. Arquitectura de contenedores .....                          | 50 |
| 5.3. Construcción de los contenedores .....                      | 51 |
| 5.4. Enrutamiento Nginx .....                                    | 52 |
| 5.5. Despliegue desde cero .....                                 | 52 |
| 5.6. Operación y mantenimiento .....                             | 53 |
| 6. Manual de uso .....                                           | 54 |
| 6.1. Acceso a la herramienta .....                               | 54 |
| 6.2. Flujo de auditoría completo .....                           | 55 |
| 6.3. Utilidades .....                                            | 61 |
| 6.4. Paneles pendientes de activación .....                      | 62 |
| 7. Conclusiones y lecciones aprendidas .....                     | 64 |
| 7.1. Lo que funcionó mejor de lo esperado .....                  | 64 |
| 7.2. Lo que fue más difícil de lo previsto .....                 | 64 |
| 7.3. Qué haríamos diferente si empezáramos de nuevo .....        | 65 |
| 7.4. Aprendizajes sobre integración de sistemas complejos .....  | 65 |
| 8. Road map de mejora para la Práctica 2 .....                   | 67 |
| 8.1. Fase 1 — Integración de módulos pendientes .....            | 67 |
| 8.2. Fase 2 — Sistema de usuarios público .....                  | 67 |
| 8.3. Fase 3 — Nuevas funcionalidades .....                       | 68 |

|                                            |    |
|--------------------------------------------|----|
| 8.4. Fase 4 — Seguridad y producción ..... | 68 |
| 8.5. Tabla resumen .....                   | 69 |

## SECCIÓN 1

## Resumen ejecutivo

---

**HookSuite** es una herramienta de auditoría de seguridad web. Su objetivo es automatizar el proceso de detección de vulnerabilidades web combinando análisis de tráfico HTTP en tiempo real, fuzzing automatizado de parámetros y análisis inteligente mediante inteligencia artificial.

Funcionalmente, HookSuite opera de forma similar a Burp Suite pero con tres diferencias clave:

- Está construida desde cero, es accesible desde cualquier navegador sin instalación de software adicional en el cliente, e incorpora inteligencia artificial para clasificar y priorizar vulnerabilidades según el estándar OWASP.

El sistema está compuesto por **cinco módulos** integrados:

- un **dashboard web en React** que actúa como interfaz de control.
- un **backend en FastAPI** que gestiona las sesiones de auditoría y ejecuta las peticiones HTTP por el auditor.
- un **motor de automatización con Playwright** para la ejecución de ataques con navegador real.
- un **módulo de captura de tráfico** basado en Chrome DevTools Protocol.
- un **módulo de inteligencia artificial** para el análisis y clasificación de vulnerabilidades.

Los módulos de Playwright, DevTools e IA están integrados en la arquitectura del sistema y su activación completa está planificada para la Práctica 2.

La herramienta está desplegada en un servidor Hetzner Cloud CX22 y es accesible desde cualquier navegador a través de su URL pública. En su estado actual, el Spider, el Repeater, el Intruder y las Utilidades están completamente operativos y han sido validados contra DVWA (Damn Vulnerable Web Application). El desarrollo completo, incluyendo el historial de commits, está disponible en el repositorio público de la organización Feet-Lovers en GitHub.

## SECCIÓN 2

# Análisis y comparación con otras herramientas

---

## El ecosistema actual de herramientas de auditoría web

Las herramientas de auditoría de seguridad web más utilizadas en la industria —Burp Suite, OWASP ZAP, Nikto— comparten un patrón común: requieren instalación local, generan grandes volúmenes de tráfico fácilmente detectable, y delegan en el analista la mayor parte del trabajo de interpretación de resultados. Son herramientas potentes pero con tres limitaciones estructurales que HookSuite aborda directamente.

La primera es la dependencia del analista humano. Herramientas como Burp Suite interceptan el tráfico y presentan los datos en bruto, pero la interpretación —determinar qué es una vulnerabilidad, qué severidad tiene, qué ataque ejecutar a continuación— recae íntegramente en el criterio del analista. Esto convierte la auditoría en un proceso que escala mal: más tráfico equivale a más tiempo de análisis manual.

La segunda es la accesibilidad. La mayoría de herramientas profesionales requieren configuración local del sistema operativo, instalación de certificados de seguridad y conocimientos técnicos previos para operar. Esto eleva la barrera de entrada y dificulta su uso en entornos educativos o en equipos con perfiles heterogéneos.

La tercera es la trazabilidad. Las auditorías realizadas con herramientas tradicionales generan logs dispersos, difícilmente exportables y sin estructura estandarizada. Integrar los resultados en un flujo de trabajo moderno —CI/CD, ticketing, informes automáticos— requiere desarrollo adicional específico para cada herramienta.

## La propuesta de HookSuite

HookSuite nace como respuesta a estas tres limitaciones. Su arquitectura combina tecnologías modernas para construir una solución que es a la vez accesible, inteligente y trazable.

La accesibilidad se resuelve mediante un dashboard web accesible desde cualquier navegador sin instalación ni configuración adicional. El analista dirige el proceso —introduce la URL objetivo, define el scope, lanza el spider, selecciona peticiones para el Repeater y configura los ataques en el Intruder— mientras el servidor ejecuta las operaciones pesadas sin intervención adicional del cliente.

La automatización del análisis se resuelve mediante un módulo de inteligencia artificial integrado en el sistema. Este módulo está diseñado para analizar cada paquete interceptado, identificar patrones de vulnerabilidad, generar instrucciones de ataque específicas

para el objetivo y clasificar los resultados según el estándar OWASP. El analista recibirá vulnerabilidades clasificadas y priorizadas, no datos en bruto.

La trazabilidad se resuelve mediante un sistema de registro estructurado. Cada vulnerabilidad detectada genera una ficha completa con identificador único, tipo, severidad, URL afectada, payload utilizado y recomendación de mitigación, exportable en formato JSON estándar.

## **| Relevancia en el contexto académico**

Este proyecto desarrolla competencias en cinco áreas simultáneamente: desarrollo web full-stack, seguridad ofensiva, integración de APIs de inteligencia artificial, trabajo con protocolos HTTP a bajo nivel e infraestructura en la nube. La naturaleza distribuida del sistema —cinco módulos integrados desarrollados en paralelo por cinco personas— añade una dimensión de ingeniería de software que va más allá del ejercicio técnico individual.

## SECCIÓN 3

# Arquitectura técnica

## Visión general del sistema

HookSuite está compuesto por cinco módulos independientes que se comunican a través de una capa de backend centralizada. Cada módulo fue desarrollado por un miembro del equipo de forma autónoma y se integra con el resto a través de contratos de API definidos previamente.

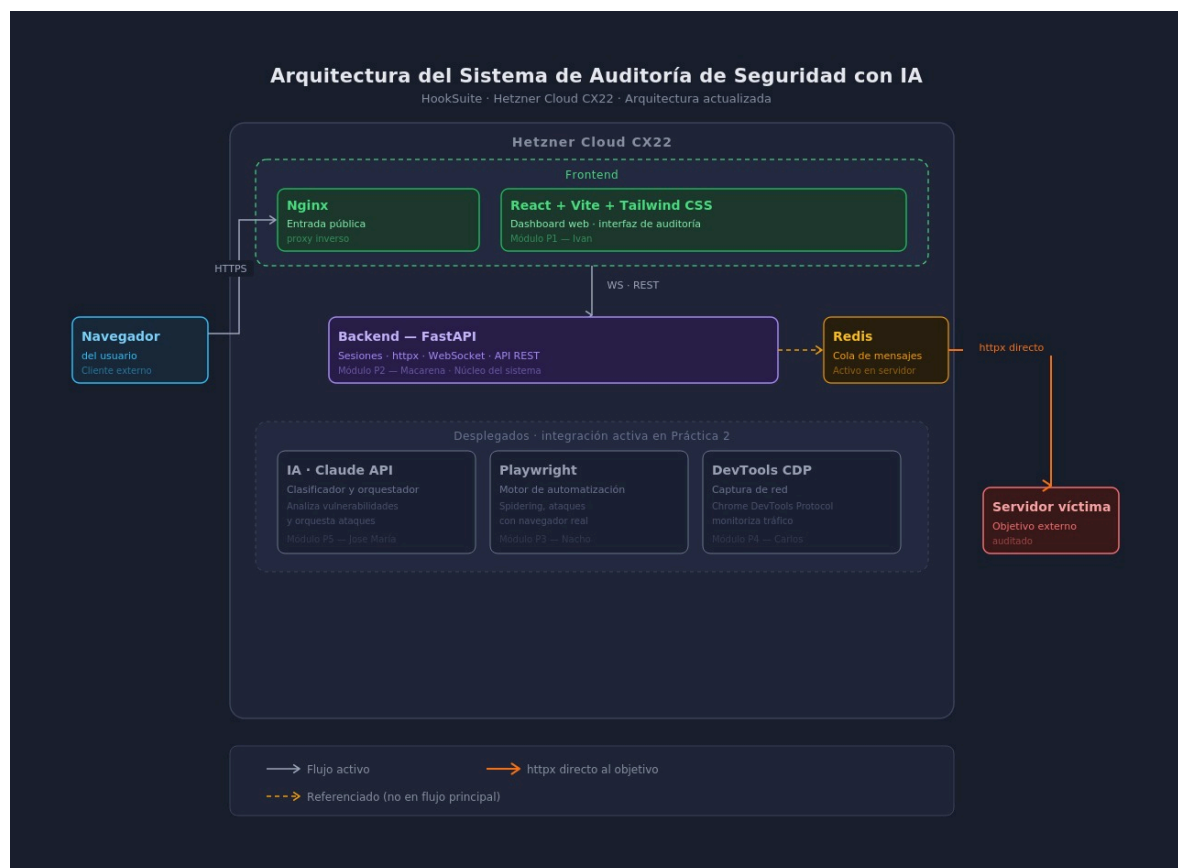


Figura 1: Arquitectura técnica de HookSuite — estado actual Práctica 1

Los módulos P3, P4 y P5 están integrados en la arquitectura y su activación completa está planificada para la Práctica 2.

## Proceso de decisión arquitectónica

El diseño de HookSuite partió de una pregunta técnica concreta: ¿cómo construir una herramienta de auditoría web accesible desde el navegador sin instalar software adicional en el cliente?

La investigación inicial identificó tres limitaciones fundamentales del navegador que condicionaron toda la arquitectura. El modelo de seguridad del navegador impide interceptar el tráfico de otras pestañas y aplicaciones por la política de mismo origen, actuar como proxy TCP al no tener acceso a sockets TCP arbitrarios, y modificar headers protegidos como Host, Origin o Cookie en las peticiones fetch y XHR. La conclusión fue que la función central de una herramienta de auditoría es técnicamente imposible si toda la lógica reside en el frontend. La solución pasaba por mover el proxy al servidor.

Con esa premisa, el equipo evaluó cinco opciones para gestionar la interceptación del tráfico:

| Opción                   | Resultado                                                |
|--------------------------|----------------------------------------------------------|
| Extensión de navegador   | Descartada — rompe el concepto de web app                |
| Certificado CA propio    | Descartada — requiere instalar certificado en el cliente |
| Proxy SOCKS5             | Descartada — configuración manual compleja               |
| Electron híbrido         | Descartada — requiere instalar aplicación de escritorio  |
| Archivo PAC + WebSockets | Seleccionada                                             |

La opción seleccionada combinaba dos elementos. Un archivo PAC (Proxy Auto-Configuration) alojado en el servidor — un fichero JavaScript que el navegador lee antes de cada petición para decidir si enrutar el tráfico a través del proxy. Y WebSockets como canal de comunicación bidireccional en tiempo real entre el frontend y el backend. Esta combinación ofrecía compatibilidad con todos los navegadores modernos, sin instalaciones en el cliente, y soporte nativo para múltiples usuarios simultáneos mediante tokens de sesión UUID.

La validez de esta arquitectura fue confirmada por Caido — una herramienta de seguridad web profesional con arquitectura cliente-servidor idéntica que también requiere configuración manual del proxy, lo que confirmó que no existe ninguna alternativa técnica viable que evite ese paso sin instalar una extensión o aplicación.

Esta arquitectura inicial fue posteriormente descartada cuando al exponerla al exterior los bots saturaron el servidor. El equipo pivotó hacia un modelo donde el servidor realiza las peticiones HTTP directamente por el auditor — eliminando la necesidad de configuración del proxy en el navegador y resolviendo el problema de saturación. Esta decisión redefinió el rol de cada módulo y condicionó la fase de integración del proyecto.

## Stack tecnológico

| Módulo   | Tecnologías                                | Responsable   |
|----------|--------------------------------------------|---------------|
| Frontend | React 19 + Vite + Tailwind CSS             | P1 — Ivan     |
| Backend  | Python 3.11 + FastAPI + httpx + WebSockets | P2 — Macarena |

|                 |                                     |                 |
|-----------------|-------------------------------------|-----------------|
| Playwright      | Python + Playwright + httpx         | P3 — Nacho      |
| DevTools        | Python + WebSockets + httpx         | P4 — Carlos     |
| IA              | Python + API Anthropic              | P5 — Jose María |
| Infraestructura | Docker + Hetzner Cloud CX22 + Nginx | Todos           |

## Contratos de integración activos — Práctica 1

| Integración                  | Canal            | Estado |
|------------------------------|------------------|--------|
| P1 ↔ P2 (Frontend — Backend) | WebSocket + REST | Activo |

Los contratos de integración de P3, P4 y P5 con el Backend se definirán y documentarán durante la Práctica 2.

## SECCIÓN 4

# Proceso de desarrollo

## Proceso individual

El desarrollo de HookSuite arrancó con cada miembro construyendo su módulo de forma autónoma. Durante esta fase el equipo trabajó en paralelo — cada uno contra su propio entorno local con DVWA como objetivo de pruebas — siguiendo los contratos de integración definidos al inicio del proyecto. El resultado fue cinco módulos independientes y validados en local, listos para conectarse entre sí.

### P1 — Frontend (Ivan Medina Castro)

#### Fase 0 — Investigación y decisiones de arquitectura

Antes de que Ivan iniciara el desarrollo, el equipo realizó una investigación técnica para definir la arquitectura de la herramienta. La conclusión fue que HookSuite operaría como un proxy en el navegador del usuario — el tráfico del auditor pasaría a través del servidor antes de llegar al objetivo, permitiendo interceptarlo y analizarlo. Esta arquitectura requería que el usuario configurara su navegador apuntando a un archivo PAC que el servidor generaba dinámicamente.

Para minimizar la fricción de esa configuración, el equipo diseñó un asistente de onboarding que detectaba automáticamente el navegador y sistema operativo del usuario y mostraba instrucciones paso a paso personalizadas. El asistente verificaba cada dos segundos si el proxy estaba activo y cerraba el modal automáticamente cuando lo detectaba. Como alternativa para usuarios que no quisieran configurar el proxy, se diseñó también un importador de peticiones que permitía pegar peticiones en formato raw HTTP o cURL directamente en el Repeater. Estas decisiones definieron el alcance inicial del módulo y los componentes que Ivan debía construir.

#### Fase 1 — Construcción con mock mode y validación con proxy PAC

Ivan construyó el Frontend completo siguiendo su manual de desarrollo, entregando la primera versión funcional con toda la estructura base del proyecto: layout con sidebar de navegación, sistema de componentes UI reutilizables, hooks de WebSocket y sesión, las seis páginas principales, el asistente de configuración del proxy con detección automática de navegador y sistema operativo, el importador de peticiones, y un sistema completo de mock data.

El sistema de mock data fue una de las decisiones técnicas más relevantes del módulo. Dado que el Frontend se desarrollaba en paralelo al Backend, Ivan construyó un conjunto de datos simulados que replicaban exactamente el formato que emitiría el WebSocket real.

Esto permitió desarrollar y validar toda la interfaz de forma completamente independiente, sin bloqueos por dependencias entre módulos.

Con la interfaz validada en mock, Ivan configuró el proxy PAC en local y conectó el Frontend contra DVWA. En esta fase la herramienta funcionó como estaba diseñada originalmente: el proxy interceptaba el tráfico del navegador del auditor, las peticiones aparecían en el panel Proxy en tiempo real, el auditor podía enviarlas al Repeater para modificarlas y reenviarlas, y el Intruder ejecutaba payloads SQLi reales contra los formularios de DVWA.

## P2 — Backend (Macarena Rogerio)

### Fase 1 — Construcción del servidor base

Macarena comenzó configurando el entorno Python en Windows, donde encontró el primer obstáculo técnico del módulo: al intentar instalar las dependencias del proyecto, la versión de Python disponible en el sistema era incompatible con `pydantic-core`, que requería compilar extensiones nativas con Rust y Cargo — una cadena de herramientas que no estaba disponible en el entorno. Lo resolvió instalando Python 3.12 mediante el gestor oficial y creando un entorno virtual específico para esa versión.

```
n (3.12)
= help: please check if an updated version of PyO3 is available. Current version: 0.21.1
= help: set PYO3_USE_ABI3_FORWARD_COMPATIBILITY=1 to suppress this check and build anyway using the s
table ABI
warning: build failed, waiting for other jobs to finish...
ōŸ'¥ maturin failed
Caused by: Failed to build a native library through cargo
Caused by: Cargo build finished with "exit code: 101": "cargo" "rustc" "--profile" "release" "--feat
ures" "pyo3/extension-module" "--message-format" "json-render-diagnostics" "--manifest-path" "C:\\Users\\c4t3
r\\AppData\\Local\\Temp\\pip-install-h55x4buq\\pydantic-core_b925644d6f76432697549be3fdf12d44\\Cargo.toml" "--
-lib" "--crate-type" "cdylib"
Error: command ['maturin', 'pep517', 'build-wheel', '-i', 'C:\\Users\\c4t3r\\AndroidStudioProjects\\Pun
tosNFC2\\Proyecto-Evolve\\venv\\Scripts\\python.exe', '--compatibility', 'off'] returned non-zero exit status
1
[End of output]

note: This error originates from a subprocess, and is likely not a problem with pip.
ERROR: Failed building wheel for pydantic-core
Failed to build pydantic-core

[notice] A new release of pip is available: 26.0.1 -> 26.1
[notice] To update, run: python.exe -m pip install --upgrade pip
error: failed-wheel-build-for-install

× Failed to build installable wheels for some pyproject.toml based projects
↳ pydantic-core
```

Figura 2: Error de compilación de `pydantic-core` durante la instalación de dependencias

Con el entorno configurado, definió la estructura de carpetas del módulo — `routes/`, `services/`, `models/`, `middleware/` — que organizaría todos los bloques funcionales del backend.



Figura 3: Estructura de carpetas del Backend planificada al inicio de la construcción

Con la estructura en su sitio, construyó el servidor FastAPI base con los endpoints de salud, la gestión de sesiones con tokens UUID y el canal WebSocket en `/ws/{token}`. Verificó el funcionamiento del servidor consultando `/health` desde el navegador y comprobando que el Swagger UI en `/docs` mostraba los endpoints registrados.

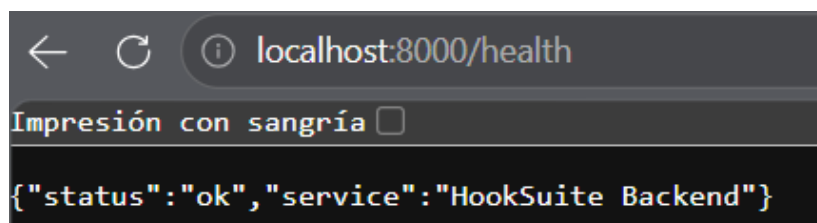


Figura 4: Endpoint `/health` respondiendo correctamente — servidor base operativo



Figura 5: Swagger UI con los primeros endpoints registrados

Una vez verificado el servidor base, añadió la gestión de sesiones con el endpoint `/api/session/new` — el punto de coordinación con el Frontend: en cuanto el WebSocket estuvo operativo, Ivan pudo conectar el Frontend al Backend real y validar el contrato de comunicación.

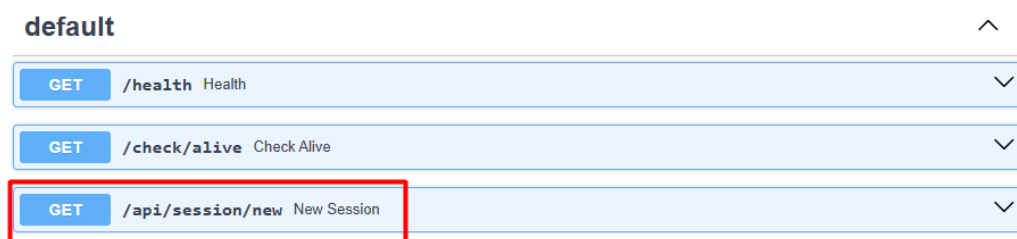


Figura 6: Swagger UI con el endpoint `/api/session/new` añadido — gestión de sesiones operativa

## Fase 2 — Implementación del proxy TCP con mitmproxy

Con el servidor base operativo, Macarena implementó la arquitectura de proxy TCP usando mitmproxy — una librería que escucha en el puerto 8080 y procesa cada petición HTTP a través de un addon personalizado que extrae los datos relevantes y los emite por WebSocket al Frontend en tiempo real. En paralelo implementó los modelos Pydantic en `schemas.py` y la función `forward_request` en el servicio de proxy HTTP, que gestiona el reenvío de peticiones con manejo de timeouts, filtrado de headers protegidos y análisis de respuestas sospechosas.

La integración de mitmproxy presentó cuatro errores encadenados que Macarena resolvió de forma sistemática: la librería no estaba instalada correctamente, una dependencia de gestión de contraseñas tenía una versión incompatible, los bloques de excepción del `proxy_service` estaban mal colocados y al corregirlos se perdieron funciones del fichero. Cada error llevó al siguiente hasta tener el servidor arrancando limpiamente con mitmproxy y FastAPI como procesos independientes.

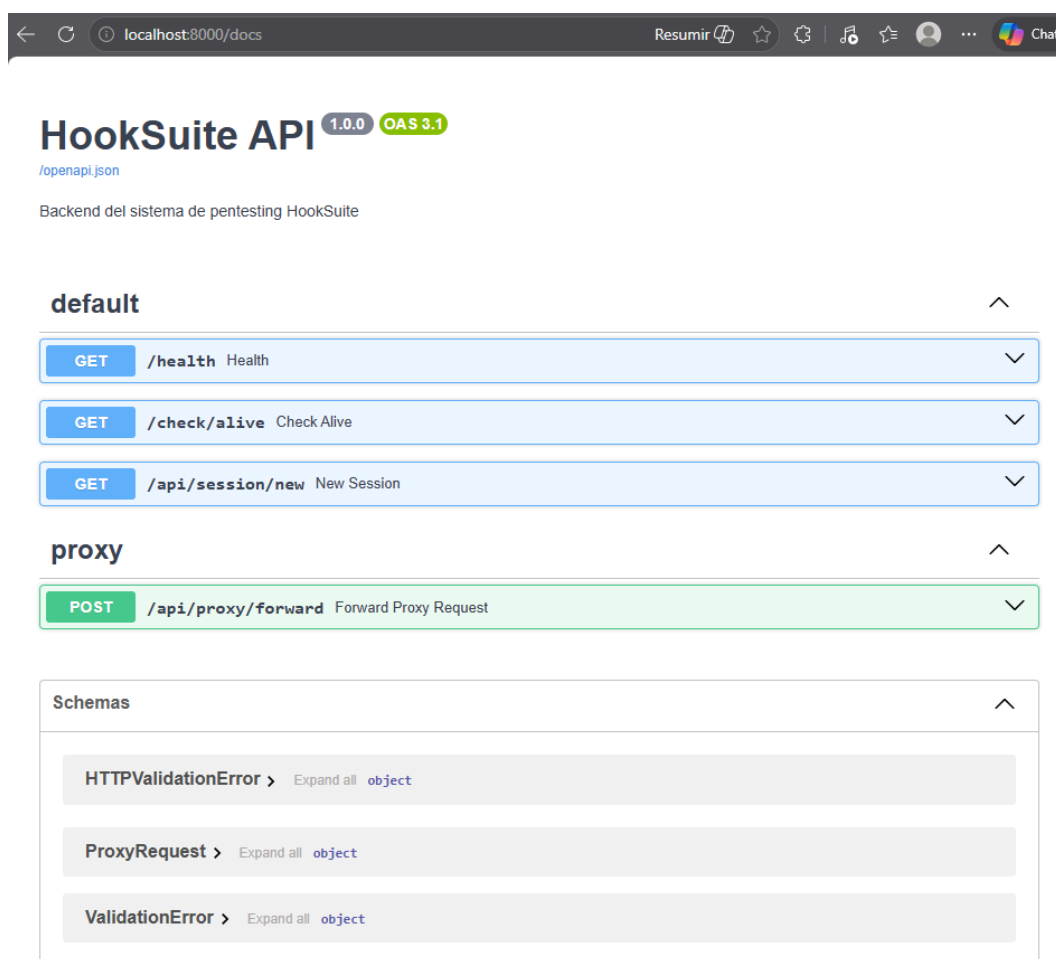


Figura 7: Swagger UI con el grupo proxy y los schemas Pydantic definidos — arquitectura de proxy interceptor operativa

### P3 — Playwright (Nacho García Monge)

#### Fase 1 — Setup y optimización de velocidad

Nacho arrancó el módulo configurando el entorno Playwright con Chromium y levantando DVWA en Docker como entorno de pruebas local.

```
PS C:\Users\nacho> docker run -d --name dvwa -p 8888:80 vulnerables/web-dvwa
Unable to find image 'vulnerables/web-dvwa:latest' locally
latest: Pulling from vulnerables/web-dvwa
eb05d18be401: Pull complete
e9968e5981d2: Pull complete
3e17c6eae66c: Pull complete
0c57df616dbf: Pull complete
2cd72dba8257: Pull complete
6cff5f35147f: Pull complete
b3d64a33242d: Pull complete
098cffd43466: Pull complete
Digest: sha256:dae203fe11646a86937bf04db0079adef295f426da68a92b40e3b181f337daa7
Status: Downloaded newer image for vulnerables/web-dvwa:latest
fb3143bdb893ed496c9ccb040ab982f8cc8a6cda159f09d84f29c6b84d82
```

Figura 8: DVWA levantándose en Docker — entorno de pruebas local operativo

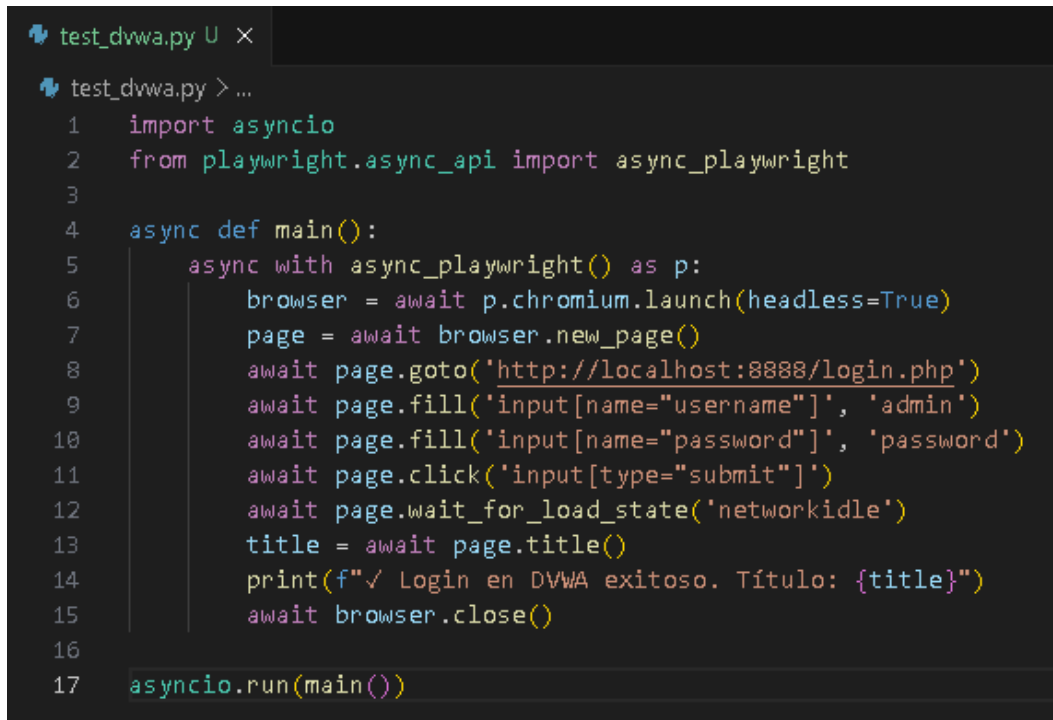
Antes de construir los componentes principales, verificó el entorno con dos scripts de prueba: uno que comprobó que Playwright arrancaba correctamente navegando a `example.com`, y otro que confirmó el login automático en DVWA. Ambos se eliminaron tras la verificación.

```
(.venv) PS C:\Users\nacho\Documents\GitHub\Proyecto-Evolve> playwright install
Downloading Chrome for Testing 147.0.7727.15 (playwright chromium v1217) from https://cdn.playwright.dev/builds/cft/147.0.7727.15/win64/chrome-win64.zip
179.4 MiB [=====] 100% 0.0s
Chrome for Testing 147.0.7727.15 (playwright chromium v1217) downloaded to C:\Users\nacho\AppData\Local\ms-playwright\chromium-1217
Downloading Chrome Headless Shell 147.0.7727.15 (playwright chromium-headless-shell v1217) from https://cdn.playwright.dev/builds/cft/147.0.7727.15/win64/chrome-headless-shell-win64.zip
111.5 MiB [=====] 100% 0.0s
Chrome Headless Shell 147.0.7727.15 (playwright chromium-headless-shell v1217) downloaded to C:\Users\nacho\AppData\Local\ms-playwright\chromium-headless-shell-1217
Downloading Firefox 148.0.2 (playwright firefox v1511) from https://cdn.playwright.dev/dbazure/download/playwright/builds/firefox/1511/firefox-win64.zip
113.1 MiB [=====] 100% 0.0s
Firefox 148.0.2 (playwright firefox v1511) downloaded to C:\Users\nacho\AppData\Local\ms-playwright\firefox-1511
Downloading Webkit 25.4 (playwright webkit v2272) from https://cdn.playwright.dev/dbazure/download/playwright/builds/webkit/2272/webkit-win64.zip
57.6 MiB [=====] 100% 0.0s
Webkit 25.4 (playwright webkit v2272) downloaded to C:\Users\nacho\AppData\Local\ms-playwright\webkit-2272
Downloading Ffmpeg (playwright ffmpeg v1011) from https://cdn.playwright.dev/dbazure/download/playwright/builds/ffmpeg/1011/ffmpeg-win64.zip
1.3 MiB [=====] 100% 0.0s
Ffmpeg (playwright ffmpeg v1011) downloaded to C:\Users\nacho\AppData\Local\ms-playwright\ffmpeg-1011
Downloading Winldd (playwright winldd v1007) from https://cdn.playwright.dev/dbazure/download/playwright/builds/winldd/1007/winldd-win64.zip
0.1 MiB [=====] 100% 0.0s
Winldd (playwright winldd v1007) downloaded to C:\Users\nacho\AppData\Local\ms-playwright\winldd-1007
```

Figura 9: Verificación de Playwright — instalación de Chromium, Firefox y WebKit completada

```
test_playwright.py U x
test_playwright.py > ...
1 import asyncio
2 from playwright.async_api import async_playwright
3
4 async def main():
5     async with async_playwright() as p:
6         browser = await p.chromium.launch(headless=True)
7         page = await browser.new_page()
8         await page.goto('https://example.com')
9         title = await page.title()
10        print(f"✓ Playwright funciona. Título de la página: {title}")
11        await browser.close()
12    asyncio.run(main())
```

Figura 10: Script de prueba ejecutándose correctamente — Playwright funciona, título de la página verificado



```
test_dwva.py U X
test_dwva.py > ...
1  import asyncio
2  from playwright.async_api import async_playwright
3
4  async def main():
5      async with async_playwright() as p:
6          browser = await p.chromium.launch(headless=True)
7          page = await browser.new_page()
8          await page.goto('http://localhost:8888/login.php')
9          await page.fill('input[name="username"]', 'admin')
10         await page.fill('input[name="password"]', 'password')
11         await page.click('input[type="submit"]')
12         await page.wait_for_load_state('networkidle')
13         title = await page.title()
14         print(f"✓ Login en DVWA exitoso. Título: {title}")
15         await browser.close()
16
17  asyncio.run(main())
```

Figura 11: Login automático en DVWA verificado — autenticación con Playwright funcionando

La primera decisión técnica fue la optimización de velocidad: verificó la diferencia real entre `page.type()` y `page.fill()` mediante un script de benchmark sobre el formulario de login de DVWA. La mejora medida — hasta 60 veces más rápido — confirmó que `page.fill()` debía usarse de forma sistemática en todo el módulo. Nacho verificó también que ningún módulo utilizara `page.type()`, confirmando que la optimización se aplicaba de forma consistente en toda la base de código.



```
PS C:\Users\nacho\Documents\GitHub\Proyecto-Evolve> Get-ChildItem -Recurse -Include *.py modules, core | Select-String -
SimpleMatch "page.type("
core\browser.py:44:      # page.fill() pasa el texto completo de una vez → 60x más rápido que page.type()
```

Figura 12: Verificación sistemática — ningún uso de `page.type()` en el código, comentario de optimización visible en `browser.py`

Con esa decisión tomada, construyó el `BrowserManager` con `asyncio.Semaphore` para el control de paralelismo, que actúa como base para todos los demás componentes.

## Fase 2 — Reconocimiento automático

Con la infraestructura base operativa, Nacho construyó los tres módulos de reconocimiento: el sistema de autenticación para DVWA y la variante genérica parametrizable, el spider con BFS y filtrado de extensiones estáticas, y el fingerprinter con detección de nueve tecnologías y generación de prioridades de ataque.

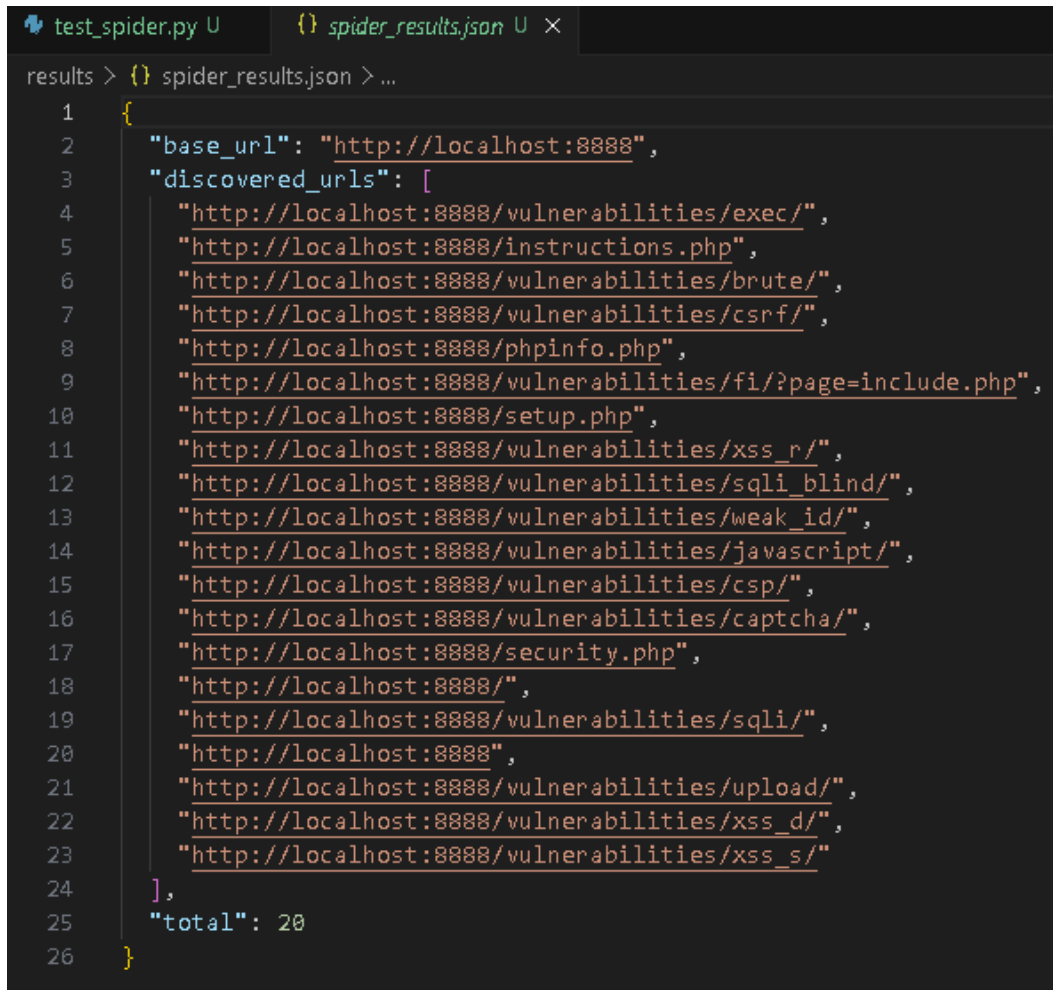
En la primera ejecución contra DVWA el spider descubrió 20 URLs — la totalidad de la superficie de ataque de la aplicación — guardando los resultados en `results/spider_results.json`.

```
• (.venv) PS C:\Users\nacho\Documents\GitHub\Proyecto-Evolve> py test_spider.py
✓ Navegador iniciado (headless=True, max_parallel=3)
✓ Login exitoso en DVWA como admin

🕷️ Iniciando spider en http://localhost:8888
Máximo de páginas: 20
→ http://localhost:8888 (20 enlaces nuevos)
→ http://localhost:8888/ (0 enlaces nuevos)
→ http://localhost:8888/instructions.php (5 enlaces nuevos)
→ http://localhost:8888/setup.php (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/brute/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/exec/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/csrf/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/fi/?page=include.php (3 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/upload/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/captcha/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/sqli/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/sqli_blind/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/weak_id/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/xss_d/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/xss_r/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/xss_s/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/csp/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/javascript/ (0 enlaces nuevos)
→ http://localhost:8888/security.php (3 enlaces nuevos)
→ http://localhost:8888/phpinfo.php (0 enlaces nuevos)

✓ Spider completado: 20 URLs descubiertas
✓ Resultados del spider guardados en results/spider_results.json
```

Figura 13: Spider ejecutándose contra DVWA — 20 URLs descubiertas en la primera ejecución



```
test_spider.py U {} spider_results.json U ×
results > {} spider_results.json > ...
1 {
2   "base_url": "http://localhost:8888",
3   "discovered_urls": [
4     "http://localhost:8888/vulnerabilities/exec/",
5     "http://localhost:8888/instructions.php",
6     "http://localhost:8888/vulnerabilities/brute/",
7     "http://localhost:8888/vulnerabilities/csrf/",
8     "http://localhost:8888/phpinfo.php",
9     "http://localhost:8888/vulnerabilities/fi/?page=include.php",
10    "http://localhost:8888/setup.php",
11    "http://localhost:8888/vulnerabilities/xss_r/",
12    "http://localhost:8888/vulnerabilities/sqli_blind/",
13    "http://localhost:8888/vulnerabilities/weak_id/",
14    "http://localhost:8888/vulnerabilities/javascript/",
15    "http://localhost:8888/vulnerabilities/csp/",
16    "http://localhost:8888/vulnerabilities/captcha/",
17    "http://localhost:8888/security.php",
18    "http://localhost:8888/",
19    "http://localhost:8888/vulnerabilities/sqli/",
20    "http://localhost:8888",
21    "http://localhost:8888/vulnerabilities/upload/",
22    "http://localhost:8888/vulnerabilities/xss_d/",
23    "http://localhost:8888/vulnerabilities/xss_s/"
24  ],
25   "total": 20
26 }
```

Figura 14: Fichero spider\_results.json generado automáticamente — 20 URLs descubiertas en formato JSON

Durante el desarrollo del fingerprinter apareció un bug que no llegó a resolverse antes de la entrega: en determinadas condiciones al procesar los headers de respuesta, el módulo lanza un error `not enough values to unpack`. El bug no bloquea el funcionamiento — cuando ocurre el fingerprinter devuelve igualmente los resultados parciales disponibles — pero es una deuda técnica identificada para la Práctica 2.

### Fase 3 — Automatización de ataques y validación del flujo completo

Con los módulos de reconocimiento operativos, Nacho implementó el motor de ataques con cuatro variantes: inyección de payloads en formularios con detección de anomalías en la respuesta, Blind SQLi boolean-based con comparación de longitudes de respuesta ante condiciones verdaderas y falsas, Blind SQLi time-based con medición de tiempos de respuesta ante payloads con `SLEEP()` y `WAITFOR DELAY`, y captura automática de screenshots como evidencia de cada ataque. El descubridor de formularios recibió también en esta fase la capacidad de detectar endpoints AJAX mediante interceptación de eventos de red.

Para validar la integración entre los módulos de reconocimiento y ataque, Nacho desarrolló `test_full_flow.py` — un test que ejecuta de forma encadenada fingerprinting, spider y ataque SQLi verificando que el receptor de instrucciones coordina correctamente los tres

módulos. Los tres tests completaron correctamente: PHP detectado, 5 URLs descubiertas, SQLi vulnerable.

```
(.venv) PS C:\Users\nacho\Documents\GitHub\Proyecto-Evolve> py test_full_flow.py
✓ Navegador iniciado (headless=True, max_parallel=3)
✓ Login exitoso en DVWA como admin

🟢 Test 1: Fingerprinting
X Error en fingerprinting de http://localhost:8888: not enough values to unpack (expected 2, got 1)
✓ Tecnologías detectadas en http://localhost:8888: ['PHP']
  Tecnologías: ['PHP']

🟢 Test 2: Spider

🕸 Iniciando spider en http://localhost:8888
  Máximo de páginas: 5
  → http://localhost:8888 (20 enlaces nuevos)
  → http://localhost:8888/ (0 enlaces nuevos)
  → http://localhost:8888/instructions.php (5 enlaces nuevos)
  → http://localhost:8888/setup.php (0 enlaces nuevos)
  → http://localhost:8888/vulnerabilities/brute/ (0 enlaces nuevos)

✓ Spider completado: 5 URLs descubiertas
✓ Resultados del spider guardados en results/spider_results.json
  URLs descubiertas: 5

🟢 Test 3: Ataque SQLi
  Vulnerable: True
  Anomalías: []

✓ Todos los tests completados
```

Figura 15: test\_full\_flow.py — los tres tests completados: fingerprinting PHP, spider 5 URLs, SQLi vulnerable: True

Con todos los módulos contruidos, Nacho los integró en el orquestador main.py y ejecutó el flujo completo contra DVWA: autenticación, fingerprinting, reconocimiento con el spider, descubrimiento de formularios y ataques automatizados. La auditoría completó las cinco fases descubriendo 15 URLs, analizando 7 formularios y detectando 1 vulnerabilidad de SQL Injection.

```
(.venv) PS C:\Users\nacho\Documents\GitHub\Proyecto-Evolve> py main.py

=====
HookSuite – Auditoría automática
Objetivo: http://localhost:8888
=====

⚠ Backend no disponible en http://localhost:8888. Guardando resultados en local.
✓ Navegador iniciado (headless=True, max_parallel=3)

📁 FASE 1: Autenticación
✓ Login exitoso en DVWA como admin

📁 FASE 2: Fingerprinting
X Error en fingerprinting de http://localhost:8888: not enough values to unpack (expected 2, got 1)
✓ Tecnologías detectadas en http://localhost:8888: ['PHP']
Prioridades de ataque: ['sqli', 'blind_sqli', 'file_inclusion']

📁 FASE 3: Reconocimiento (Spider)

🕸 Iniciando spider en http://localhost:8888
Máximo de páginas: 15
→ http://localhost:8888 (20 enlaces nuevos)
→ http://localhost:8888/ (0 enlaces nuevos)
→ http://localhost:8888/instructions.php (5 enlaces nuevos)
→ http://localhost:8888/setup.php (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/brute/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/exec/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/csrf/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/fi/?page=include.php (3 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/upload/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/captcha/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/sqli/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/sqli_blind/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/weak_id/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/xss_d/ (0 enlaces nuevos)
→ http://localhost:8888/vulnerabilities/xss_r/ (0 enlaces nuevos)

✓ Spider completado: 15 URLs descubiertas
✓ Resultados del spider guardados en results/spider_results.json

📁 FASE 4: Descubrimiento de formularios
→ http://localhost:8888/vulnerabilities/exec/: 1 formularios encontrados
→ http://localhost:8888/vulnerabilities/xss_r/: 1 formularios encontrados
→ http://localhost:8888/setup.php: 1 formularios encontrados
→ http://localhost:8888/vulnerabilities/xss_d/: 1 formularios encontrados
→ http://localhost:8888/vulnerabilities/csrf/: 1 formularios encontrados
→ http://localhost:8888/: 0 formularios encontrados
→ http://localhost:8888/vulnerabilities/weak_id/: 0 formularios encontrados
→ http://localhost:8888/vulnerabilities/brute/: 1 formularios encontrados
→ http://localhost:8888/vulnerabilities/fi/?page=include.php: 0 formularios encontrados
→ http://localhost:8888/vulnerabilities/captcha/: 1 formularios encontrados
```

Figura 16: Orquestador main.py ejecutando las cinco fases — autenticación, fingerprinting, spider, formularios y ataques

```
✓ Descubrimiento completado: 7 formularios en 10 páginas
✓ Formularios guardados en results/forms_discovered.json

📄 FASE 5: Ataques automatizados

🔗 Atacando campo 'ip' en http://localhost:8888/vulnerabilities/exec/

🔍 Iniciando Blind SQLi boolean en campo 'ip'
⚠️ POSIBLE Blind SQLi detectado! Diferencia: 2000 caracteres

🔗 Atacando campo 'name' en http://localhost:8888/vulnerabilities/xss_r/

🔍 Iniciando Blind SQLi boolean en campo 'name'
✓ Sin indicios de Blind SQLi (diferencia: 0 caracteres)

🔗 Atacando campo 'user_token' en http://localhost:8888/setup.php

🔍 Iniciando Blind SQLi boolean en campo 'user_token'
✓ Sin indicios de Blind SQLi (diferencia: 0 caracteres)

🔗 Atacando campo 'default' en http://localhost:8888/vulnerabilities/xss_d/

🔍 Iniciando Blind SQLi boolean en campo 'default'
✓ Sin indicios de Blind SQLi (diferencia: 0 caracteres)

🔗 Atacando campo 'password_new' en http://localhost:8888/vulnerabilities/csrf/

🔍 Iniciando Blind SQLi boolean en campo 'password_new'
✓ Sin indicios de Blind SQLi (diferencia: 0 caracteres)

🔗 Atacando campo 'password_conf' en http://localhost:8888/vulnerabilities/csrf/

🔍 Iniciando Blind SQLi boolean en campo 'password_conf'
✓ Sin indicios de Blind SQLi (diferencia: 0 caracteres)
✓ Resultados de ataque guardados en results/attack_results.json
✓ Log local guardado en results/events_log.json (2 eventos)

=====
AUDITORÍA COMPLETADA
URLs descubiertas: 15
Formularios analizados: 7
Vulnerabilidades encontradas: 1
=====
```

Figura 17: AUDITORÍA COMPLETADA — 15 URLs descubiertas, 7 formularios analizados, 1 vulnerabilidad encontrada

Desarrolló también `demo.py` como script de demostración del flujo completo, que confirmó la detección de SQLi mediante el error `SQL_ERROR: you have an error in your sql syntax` en la respuesta.

```

• (.venv) PS C:\Users\nacho\Documents\GitHub\Proyecto-Evolve> py demo.py

=====
HookSuite — DEMO de auditoría automatizada
Objetivo: DVWA (Damn Vulnerable Web Application)
=====
⚠ Backend no disponible en http://localhost:8000. Guardando resultados en local.

[PASO 1] Iniciando navegador...
✓ Navegador iniciado (headless=True, max_parallel=3)
[PASO 2] Autenticándose en DVWA...
✓ Login exitoso en DVWA como admin
[PASO 3] Detectando tecnologías del objetivo...
X Error en fingerprinting de http://localhost:8888: not enough values to unpack (expected 2, got 1)
✓ Tecnologías detectadas en http://localhost:8888: ['PHP']
  → Tecnologías detectadas: ['PHP']
  → Prioridades de ataque: ['sqli', 'blind_sqli', 'file_inclusion']

[PASO 4] Mapeando la aplicación web (spider)...

🕷 Iniciando spider en http://localhost:8888
Máximo de páginas: 8
  → http://localhost:8888 (20 enlaces nuevos)
  → http://localhost:8888/ (0 enlaces nuevos)
  → http://localhost:8888/instructions.php (5 enlaces nuevos)
  → http://localhost:8888/setup.php (0 enlaces nuevos)
  → http://localhost:8888/vulnerabilities/brute/ (0 enlaces nuevos)
  → http://localhost:8888/vulnerabilities/exec/ (0 enlaces nuevos)
  → http://localhost:8888/vulnerabilities/csrf/ (0 enlaces nuevos)
  → http://localhost:8888/vulnerabilities/fi/?page=include.php (3 enlaces nuevos)

✓ Spider completado: 8 URLs descubiertas
  → 8 URLs descubiertas automáticamente

[PASO 5] Descubriendo formularios vulnerables...
  → http://localhost:8888/vulnerabilities/sqli/: 1 formularios encontrados
  → 1 formularios encontrados en la página SQLi

[PASO 6] Lanzando ataque SQL Injection...
  → Probando payload: ' OR '1'='1
  → Probando payload: ' OR '1'='1'--

⚠ VULNERABILIDAD DETECTADA!
  → Tipo: SQL Injection
  → Payload: ' OR '1'='1'--
  → Anomalías: ['SQL_ERROR: you have an error in your sql syntax']
  → Captura guardada: results/screenshots/after_id_1778150163.png

[PASO 7] Probando Blind SQLi...

🔍 Iniciando Blind SQLi boolean en campo 'id'
✓ Sin indicios de Blind SQLi (diferencia: 0 caracteres)
  → Sin indicios de Blind SQLi
✓ Log local guardado en results/events_log.json (1 eventos)
✓ Resultados de ataque guardados en results/attack_results.json

```

Figura 18: demo.py — flujo completo en 7 pasos, SQL Injection detectada con error SQL real en la respuesta

## P4 — DevTools (Carlos Bañuelos Fernández)

### Fase 1 — Construcción del módulo

Carlos construyó el módulo de forma completamente autónoma. La primera decisión técnica fue usar el CDP de forma nativa mediante WebSocket en lugar de una librería de alto nivel — necesitaba control total sobre qué eventos capturar y cuándo recuperar el body de la respuesta. Con esa decisión tomada construyó los cinco componentes del módulo: el lanzador de Chrome con detección multiplataforma, el cliente CDP con su sistema de comandos asíncronos, el constructor de paquetes con filtrado de tráfico irrelevante, los analizadores de red y consola, y el reporter con buffer local como fallback.

```

cyber@joSeMa MINGW64 ~/Proyecto-Evolve/devtools (develop)
$ cd ~/Proyecto-Evolve/devtools
taskkill //F //IM chrome.exe //T 2>>dev/null; python -c "import asyncio; from main import run_capture; asyncio.run(run_capture('http://91.98.143.219:8888', duration_seconds=3600))"

[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (2ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (388ms)
[GET] http://91.98.143.219:8888/vulnerabilities/brute/ → 200 (762ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (4ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (444ms)
[GET] http://91.98.143.219:8888/vulnerabilities/exec/ → 200 (937ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (3ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (414ms)
[GET] http://91.98.143.219:8888/vulnerabilities/sqli/ → 200 (908ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (7ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (413ms)
[GET] http://91.98.143.219:8888/vulnerabilities/sqli/?id=1&Submit=Submit → 200 (892ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (6ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (410ms)
[GET] http://91.98.143.219:8888/ → 200 (910ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (8ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (425ms)
[GET] http://91.98.143.219:8888/vulnerabilities/sqli/ → 200 (898ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (7ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (420ms)
[GET] http://91.98.143.219:8888/vulnerabilities/sqli/?id=1&Submit=Submit → 200 (955ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (7ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (415ms)
[GET] http://91.98.143.219:8888/ → 200 (909ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (8ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (399ms)
[GET] http://91.98.143.219:8888/vulnerabilities/brute/ → 200 (887ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (7ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (407ms)
[GET] http://91.98.143.219:8888/vulnerabilities/sqli/ → 200 (961ms)
[GET] http://91.98.143.219:8888/dvwa/js/dvwaPage.js → 200 (7ms)
[GET] http://91.98.143.219:8888/dvwa/js/add_event_listeners.js → 200 (474ms)
[GET] http://91.98.143.219:8888/vulnerabilities/sqli/?id=1&Submit=Submit → 200 (961ms)

```

Figura 19: Módulo DevTools en ejecución — capturando peticiones HTTP de DVWA vía Chrome DevTools Protocol

## Fase 2 — Validación contra DVWA

Con el módulo construido, Carlos lo validó contra DVWA verificando que el CDP capturaba correctamente las peticiones HTTP, que el filtrado eliminaba el tráfico irrelevante — imágenes, fuentes, CDNs — dejando únicamente las peticiones relevantes, y que los analizadores detectaban correctamente los patrones sospechosos al provocar errores SQL en los formularios de DVWA.

En esta fase apareció el bug más relevante del módulo: Chrome ignoraba los flags de debugging cuando ya había una instancia abierta en el sistema. Carlos lo identificó y lo resolvió añadiendo `--user-data-dir` con un directorio de perfil separado, garantizando que el Chrome lanzado por DevTools arranque siempre con los flags correctos independientemente del estado del sistema. También añadió `--remote-allow-origins=*` para evitar restricciones de origen en la conexión WebSocket al CDP.

```

HookSuite — Captura Chrome DevTools
Objetivo: http://localhost:8888 | Duración: 60s

⚠ Backend no disponible. Guardando en local.
Lanzando Chrome con debugging activo...
✓ Chrome lanzado con debugging en puerto 9222 (PID: 8941)
✓ CDP activo. 1 pestaña(s) disponible(s)
  → Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development* — http://localhost:8888/login.php
✓ Conectado al CDP: Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*
✓ Network domain activado
✓ Page domain activado
✓ Analizador de red activo
✓ Console domain activado
✓ Analizador de consola activo

✓ Captura activa. Navega en Chrome para capturar tráfico.
Duración máxima: 60s | Presiona Ctrl+C para detener.

```

Figura 20: Arranque del módulo DevTools — Chrome lanzado, CDP activo y analizadores inicializados

```

✓ Captura activa. Navega en Chrome para capturar tráfico.
  Duración máxima: 60s | Presiona Ctrl+C para detener.

[GET] http://localhost:8888/index.php → 200 (68ms)
[GET] http://localhost:8888/dvwa/js/dvwaPage.js → 200 (31ms)
[GET] http://localhost:8888/dvwa/js/add_event_listeners.js → 200 (33ms)
⚠ Sospechoso: http://localhost:8888/vulnerabilities/sql/?id=276Submit=Su → ['SQL_ERROR_IN_RESPONSE']
⚠ [GET] http://localhost:8888/vulnerabilities/sql/?id=276Submit=Submit → 200 (54ms)

```

Figura 21: Captura de tráfico en tiempo real — petición marcada como sospechosa con SQL\_ERROR\_IN\_RESPONSE

```

[GET] http://localhost:8888/index.php → 200 (78ms)
[GET] http://localhost:8888/dvwa/js/dvwaPage.js → 200 (41ms)
[GET] http://localhost:8888/dvwa/js/add_event_listeners.js → 200 (41ms)
🔍 Consola [ERROR]: ['PASSWORD_EXPOSED'] → password=admin123
🔍 CONSOLA [ERROR]: ['PASSWORD_EXPOSED']
[GET] http://localhost:8888/dvwa/js/dvwaPage.js → 200 (24ms)
[GET] http://localhost:8888/dvwa/js/add_event_listeners.js → 200 (25ms)
[GET] http://localhost:8888/vulnerabilities/xss_r/?name=%3Cscript%3Econsole. → 200 (59ms)

```

Figura 22: Analizador de consola detectando PASSWORD\_EXPOSED en los logs de DVWA

```

{
  "id": "8c870071-1ca8-4308-a81d-cbe91c274423",
  "method": "GET",
  "url": "http://localhost:8888/index.php",
  "status": 200,
  "size": 6808,
  "time": 68,
  "timestamp": "2026-05-10T16:29:16.182000",
  "request_body": null,
  "suspicious": false,
  "vulnerable": false,
  "security_issues": [
    "MISSING_HEADER: content-security-policy",
    "MISSING_HEADER: x-frame-options",
    "MISSING_HEADER: strict-transport-security",
    "MISSING_HEADER: x-content-type-options",
    "MISSING_HEADER: referrer-policy"
  ]
}

```

Figura 23: Objeto paquete generado por el constructor — cinco headers de seguridad ausentes detectados

## P5 — IA (Jose María López Ausín)

### Fase 1 — Construcción del módulo

El módulo arrancó con la construcción del cliente, los cuatro prompts especializados y el clasificador. La primera decisión técnica fue el formato JSON obligatorio en todos los prompts — la alternativa era parsear texto libre, pero eso introduce fragilidad que no tiene cabida en una herramienta de auditoría donde la precisión es crítica.

Con la estructura base lista, se validó el ciclo completo contra la API real mediante un test standalone sobre DVWA en local: el prompt de paquetes de red detectó una inyección SQL con un 95% de confianza en el primer intento, sin necesidad de que el Backend ni Playwright estuvieran activos.

```

MINIOWM4\Users\Cyber\Proyecto-Evolve
(cero)
cero@MINIOWM4 MINGW64 ~/Proyecto-Evolve (feature/14)
$ python -m ha.tests.test-standalone
HookSuite - Test standalone módulo IA
Conectado con Claude API...

TEST 1 - Análisis de paquete con SQLi
=====
VULNERABILIDAD DETECTADA:
{
  "origen": "Proxy",
  "url": "http://www.local/vulnerabilidades/sqli/?id=1"~"1&Submit=Submit",
  "tipo": "SQLi",
  "severidad": "Crítica",
  "confianza": 95,
  "descripcion": "Inyección SQL clásica mediante bugs de autenticación con payload OR '1'='1'",
  "evidencia": "URL: id=1 OR '1'='1' | RESPONSE: 200 | OR '1'='1' seguido de múltiples registros (admin, Gordon Brown)",
  "recomendacion": "Implementar prepared statements, validar y sanitizar entrada del parámetro 'id', usar whitelists de caracteres permitidos"
}

TEST 2 - Fingerprint de aplicación
=====
FINGERPRINT DETECTADO:
{
  "servidor": "Apache",
  "framework": "PHP",
  "cms": "ninguno",
  "base_datos": "desconocido",
  "version_detectada": "Apache/2.4.41, PHP/7.4.3",
  "headers_seguridad_autentic": [
    "X-Frame-Options",
    "X-Content-Type-Options",
    "X-XSS-Protection",
    "Strict-Transport-Security",
    "Content-Security-Policy",
    "Referrer-Policy"
  ],
  "sectores_prioritarios": [
    {
      "tipo": "SQLi",
      "motivo": "PHP sin framework aparente sugiere código personalizado vulnerable, especialmente en formularios de login",
      "prioridad": 1
    },
    {
      "tipo": "XSS",
      "motivo": "Ausencia total de headers de seguridad anti-XSS y aplicación PHP básica",
      "prioridad": 2
    },
    {
      "tipo": "LFI",
      "motivo": "PHP con Apache permite inclusión de archivos locales si hay parámetros de archivo no validados",
      "prioridad": 3
    },
    {
      "tipo": "RCE",
      "motivo": "PHP 7.4.3 puede tener vulnerabilidades conocidas y funciones peligrosas habilitadas",
      "prioridad": 4
    }
  ],
  "confianza": 85
}

Tests completados
(cero)
cero@MINIOWM4 MINGW64 ~/Proyecto-Evolve (feature/14)

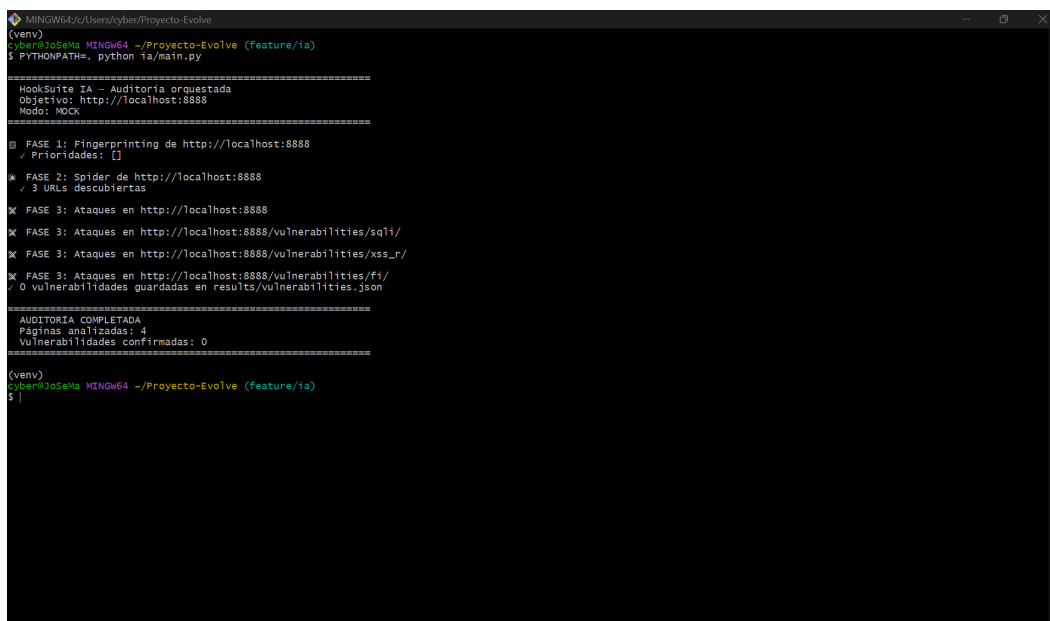
```

Figura 24: Test standalone — detección de SQLi con 95% de confianza y fingerprint con vectores priorizados

Durante esta fase apareció una incidencia de seguridad: la API key se expuso accidentalmente en el fichero `.env.example` subido al repositorio. GitHub Secret Scanning la detectó y revocó automáticamente. La versión `0.25.0` de la librería de Anthropic era además incompatible con Python 3.14 del sistema, lo que se resolvió actualizando a `>=0.97.0`.

## Fase 2 — Orquestador y modo mock

Con el clasificador validado, se construyó el orquestador con las tres fases de auditoría y el modo mock controlado por variable de entorno. El modo mock fue una decisión de diseño deliberada: permitía desarrollar y probar el ciclo completo IA→Playwright→Backend sin depender de que los otros módulos estuvieran operativos. Durante la construcción del orquestador aparecieron dos bugs que se resolvieron antes del primer commit: algunos métodos síncronos del clasificador se llamaban con `await`, y la firma del método `fingerprint` era incorrecta.



```
MINGW64/c/Users/cyber/Proyecto-Evolve
(venv)
cyber@305Ma MINGW64 ~/Proyecto-Evolve (feature/ia)
$ PYTHONPATH=. python ia/main.py

=====
HookSuite IA - Auditoria orquestada
Objetivo: http://localhost:8888
Modo: MOCK
=====

[ ] FASE 1: Fingerprinting de http://localhost:8888
  / Prioridades: [ ]
[*] FASE 2: Spider de http://localhost:8888
  / 3 URLs descubiertas
[*] FASE 3: Ataques en http://localhost:8888
[*] FASE 3: Ataques en http://localhost:8888/vulnerabilities/sqli/
[*] FASE 3: Ataques en http://localhost:8888/vulnerabilities/xss_r/
[*] FASE 3: Ataques en http://localhost:8888/vulnerabilities/ff/
  / 0 vulnerabilidades guardadas en results/vulnerabilities.json
=====
AUDITORIA COMPLETADA
Páginas analizadas: 4
Vulnerabilidades confirmadas: 0
=====

(venv)
cyber@305Ma MINGW64 ~/Proyecto-Evolve (feature/ia)
$ |
```

Figura 25: Orquestador en modo mock — las tres fases ejecutadas, 4 URLs descubiertas, 0 vulnerabilidades confirmadas (comportamiento esperado en mock)

## Integración

### Integración en local

Con los cinco módulos construidos y validados de forma independiente, el equipo se reunió para conectarlos por primera vez. La integración en local se realizó con el sistema completo corriendo en los equipos de desarrollo — Backend, Frontend, módulo IA, Playwright y DevTools levantados simultáneamente, con DVWA como objetivo de pruebas.

La integración fue fluida. Los contratos de API definidos al inicio del proyecto habían eliminado la mayoría de los problemas de compatibilidad — cuando el Frontend conectó al WebSocket real del Backend los datos fluían con el formato esperado, el módulo IA operaba en modo mock coordinándose con el Backend, Playwright respondía en modo polling y DevTools capturaba tráfico y lo enviaba al panel Red. Los ajustes necesarios fueron menores: correcciones de puerto, apuntado de conexiones entre servicios y pequeños ajustes de configuración. Al cierre de esta fase el sistema funcionaba end-to-end en local con todos los módulos activos: el proxy interceptaba el tráfico del navegador del auditor, las peticiones aparecían en el panel Proxy en tiempo real, el Repeater reenviaba peticiones modificadas, el Intruder ejecutaba ataques SQLi contra DVWA con resultados visibles en el panel de Vulnerabilidades, el panel Red mostraba el tráfico capturado por DevTools y el módulo IA analizaba los paquetes en modo mock.

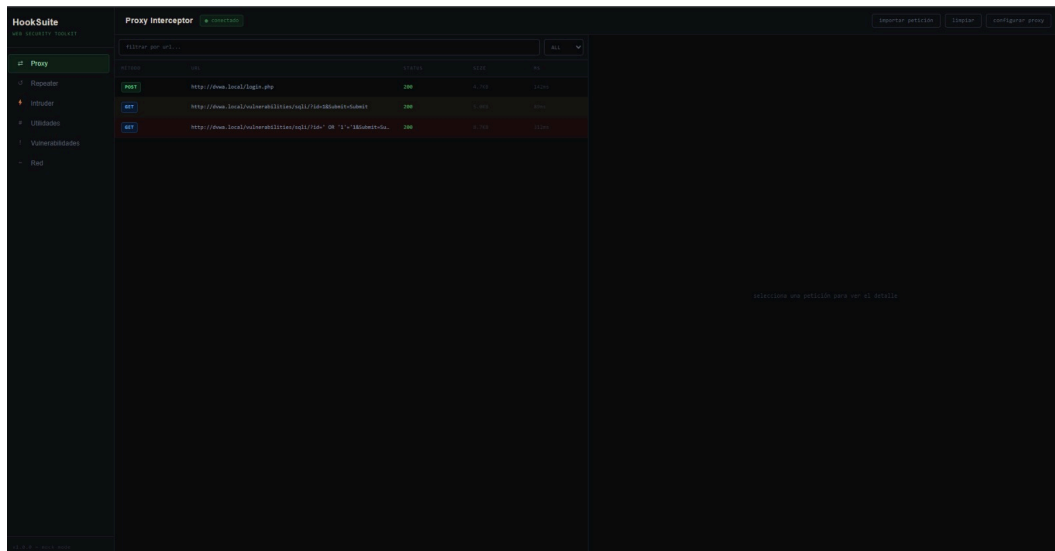


Figura 26: Panel Proxy interceptando tráfico real de DVWA en local — peticiones con status codes reales

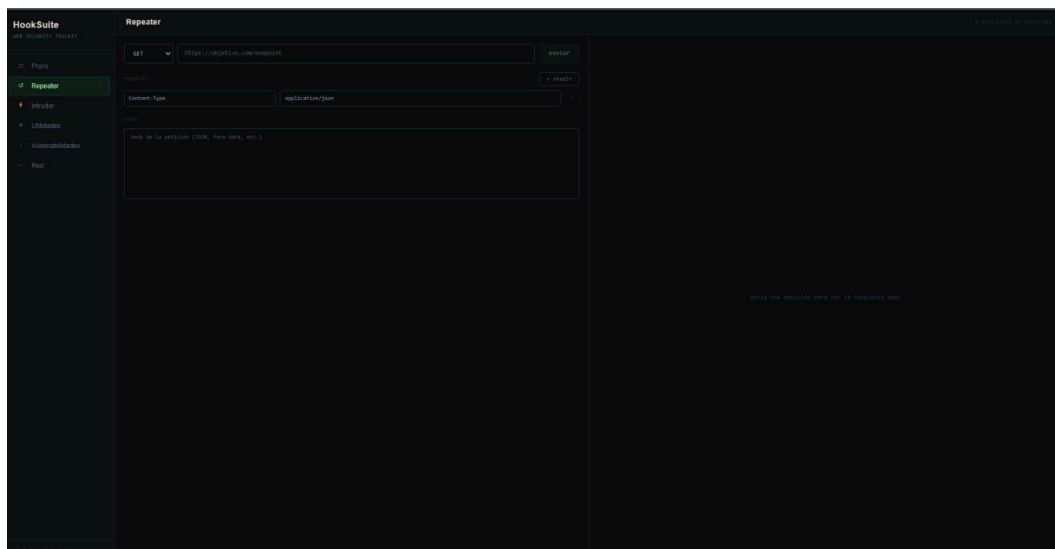


Figura 27: Panel Repeater en local — editor de peticiones con soporte de headers y body

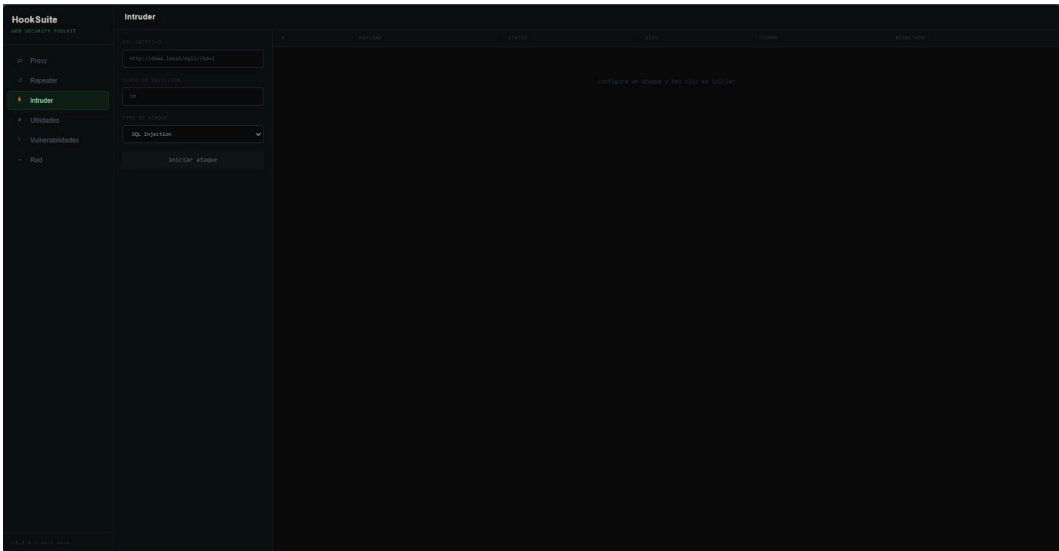


Figura 28: Panel Intruder en local — URL objetivo, punto de inyección y tipo de ataque configurados

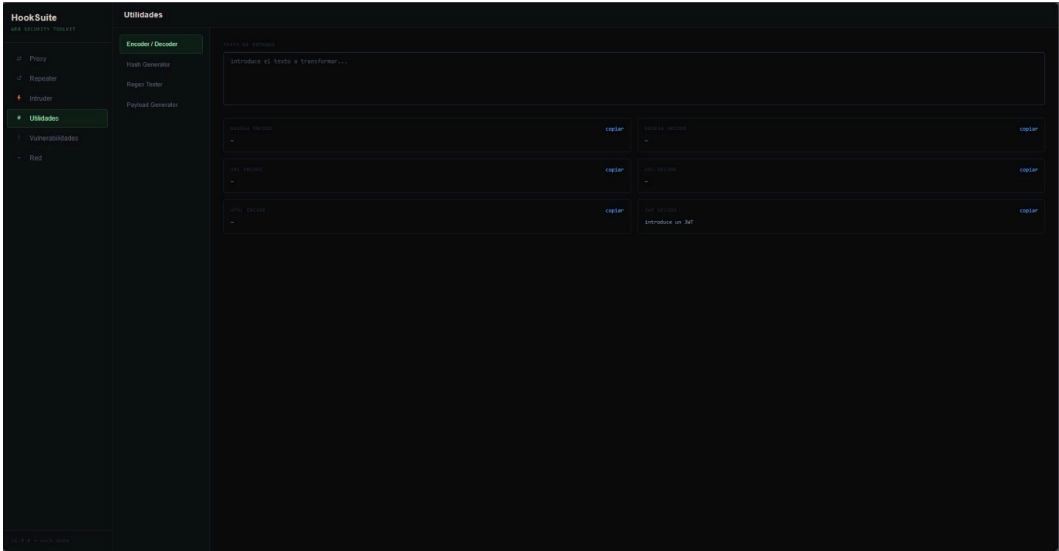


Figura 29: Panel Utilidades en local — Encoder/Decoder con todos los formatos disponibles

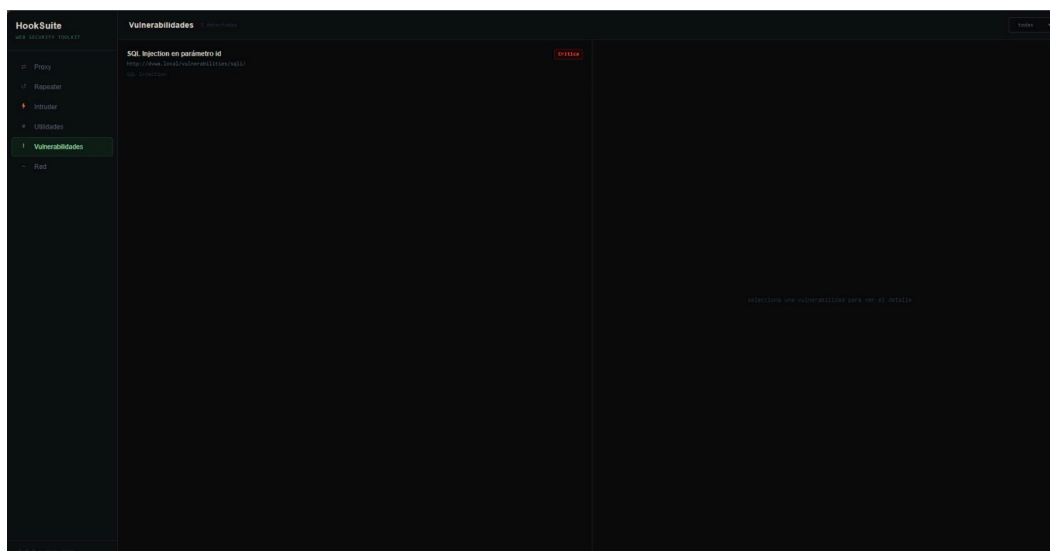


Figura 30: Panel Vulnerabilidades en local — SQL Injection crítica detectada en DVWA

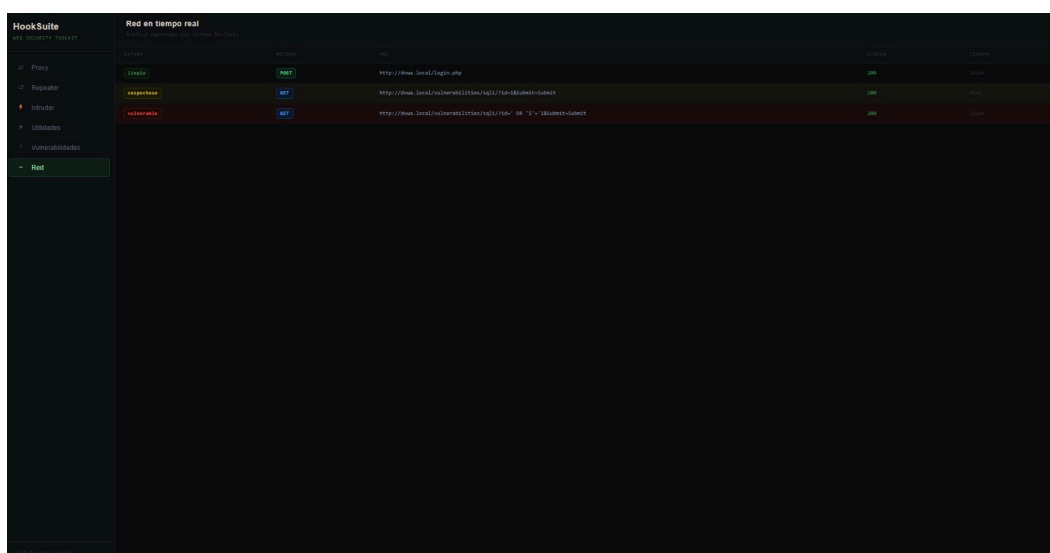


Figura 31: Panel Red en local — tráfico clasificado como limpio, sospechoso y vulnerable

## Despliegue en Hetzner

Con el sistema validado en local, el equipo preparó el repositorio para producción y creó el servidor. Macarena generó los Dockerfiles de todos los módulos y el `docker-compose.yml` de producción, añadió el `.env.example` con todas las variables del sistema y configuró los endpoints de integración. Nacho adaptó el modo de arranque del contenedor de Playwright a polling pasivo para que el sistema pudiera desplegarse sin bloquear el arranque del resto de servicios. Ivan corrigió los nombres de ficheros con mayúsculas incorrectas en Linux y el bug de `crypto.randomUUID` no disponible en HTTP.

Con el repositorio listo, se aprovisionó el servidor — un CX22 con Ubuntu 24.04 en Hetzner Cloud, IP `www.hooksuite.de` — se instaló Docker y se ejecutó `docker compose up -d --build`. El despliegue presentó cinco bugs que se resolvieron en tiempo real: `node_modules` subido al repositorio, el `docker-compose.yml` malformado, nombres de ficheros con mayúsculas incorrectas en Linux, `crypto.randomUUID` no disponible en HTTP y la URL del PAC apuntando

a localhost en lugar de a la IP pública del servidor. Con los cinco resueltos, HookSuite quedó accesible desde internet en <http://www.hooksuite.de> con el dashboard funcionando y el proxy conectado.

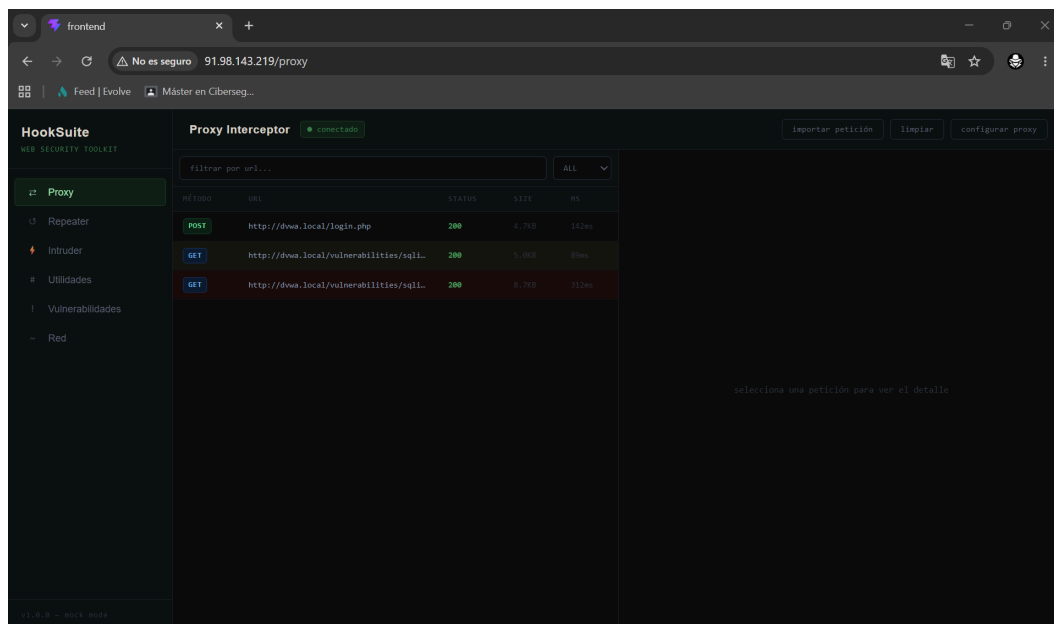


Figura 32: Dashboard de HookSuite accesible desde internet — sistema desplegado en Hetzner

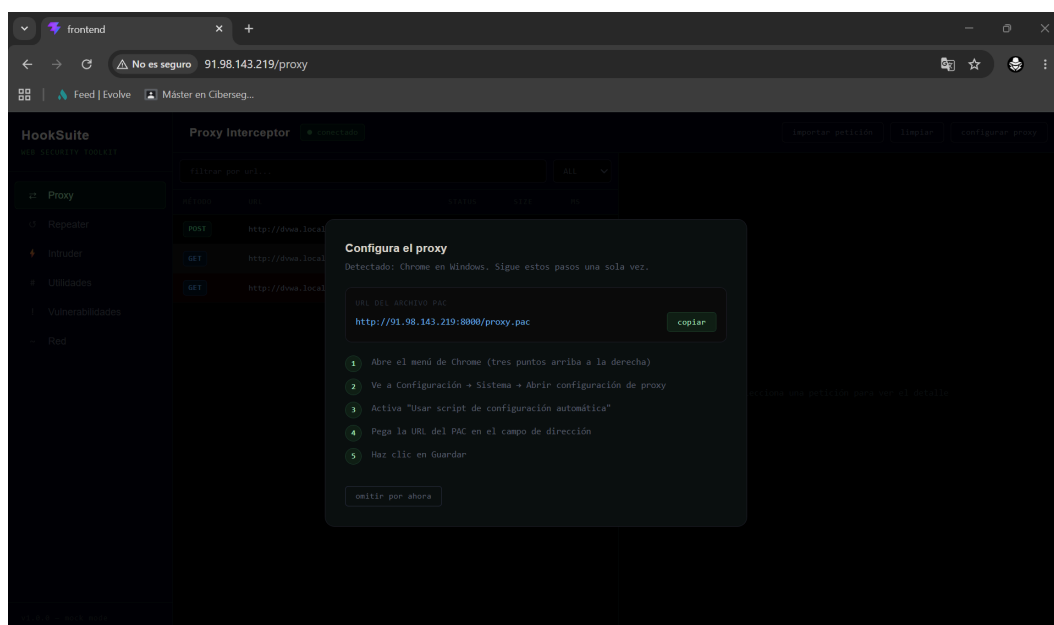


Figura 33: Asistente de configuración del proxy PAC apuntando a la IP pública del servidor

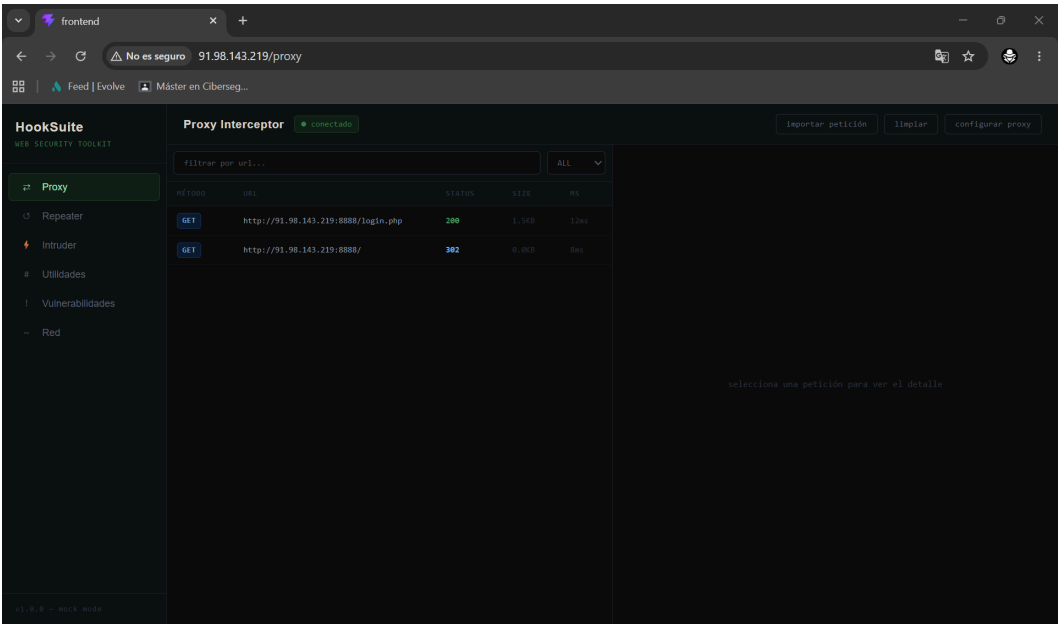


Figura 34: Panel Proxy interceptando tráfico real en el servidor de producción

| Request                   | Response                                                                                                        | Headers | enviar al repeater → |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|---------|----------------------|
| HEADERS DE PETICIÓN       |                                                                                                                 |         |                      |
| Host                      | 91.98.143.219:8888                                                                                              |         |                      |
| Proxy-Connection          | keep-alive                                                                                                      |         |                      |
| Upgrade-Insecure-Requests | 1                                                                                                               |         |                      |
| User-Agent                | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36 |         |                      |
| Accept                    | text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image-exchange;v=b3;q=0.7                      |         |                      |
| Referer                   | http://91.98.143.219:8888/vulnerabilities/sqli/                                                                 |         |                      |
| Accept-Encoding           | gzip, deflate                                                                                                   |         |                      |
| Accept-Language           | es-ES,es;q=0.9                                                                                                  |         |                      |
| Cookie                    | PHPSESSID=t5eu440jp51auc5ni3mc3du687; security=low                                                              |         |                      |
| X-HookSuite-ID            | 59306ed3-84bc-4f13-b427-91b3d5435f70                                                                            |         |                      |
| HEADERS DE RESPUESTA      |                                                                                                                 |         |                      |
| Date                      | Sun, 17 May 2026 12:03:37 GMT                                                                                   |         |                      |
| Server                    | Apache/2.4.25 (Debian)                                                                                          |         |                      |
| Expires                   | Tue, 23 Jun 2009 12:00:00 GMT                                                                                   |         |                      |
| Cache-Control             | no-cache, must-revalidate                                                                                       |         |                      |
| Pragma                    | no-cache                                                                                                        |         |                      |
| Vary                      | Accept-Encoding                                                                                                 |         |                      |
| Content-Encoding          | gzip                                                                                                            |         |                      |

Figura 35: Detalle de headers — cookie PHPSESSID de sesión autenticada visible

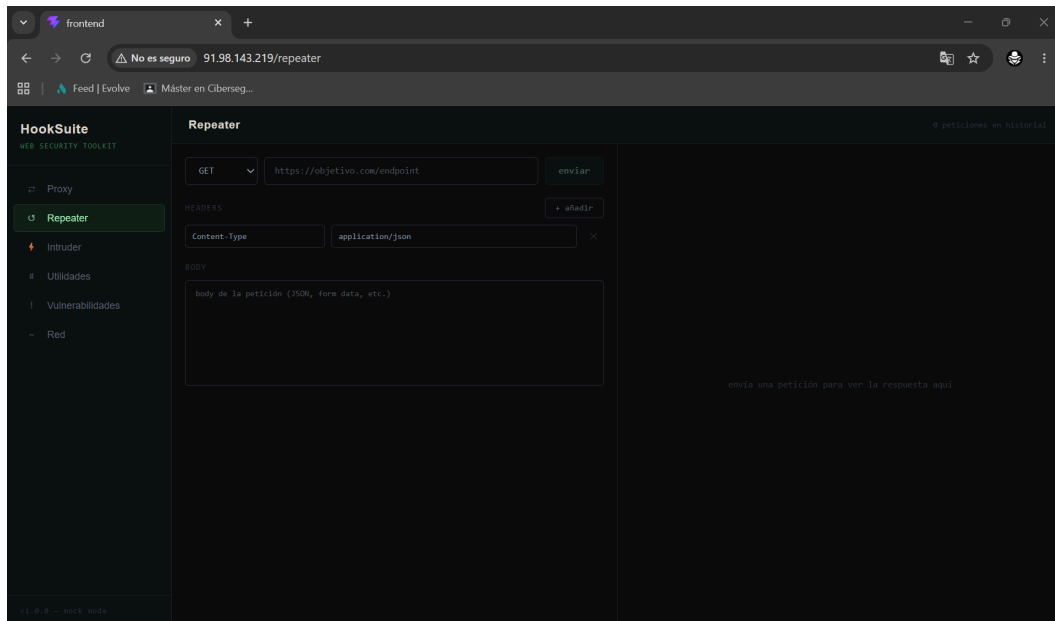


Figura 36: Panel Repeater operativo en producción

### Repeater

GET ▼  enviar

HEADERS + añadir

|                           |                                                       |   |
|---------------------------|-------------------------------------------------------|---|
| Host                      | 91.98.143.219:8888                                    | × |
| Proxy-Connection          | keep-alive                                            | × |
| Upgrade-Insecure-Requests | 1                                                     | × |
| User-Agent                | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit | × |
| Accept                    | text/html,application/xhtml+xml,application/xml;q=0.9 | × |
| Referer                   | http://91.98.143.219:8888/vulnerabilities/sqli/       | × |
| Accept-Encoding           | gzip, deflate                                         | × |
| Accept-Language           | es-ES,es;q=0.9                                        | × |
| Cookie                    | PHPSESSID=t5eu440jp51auc5ni3mc3du687; security=low    | × |
| X-HookSuite-ID            | 3999704a-d9ce-4a2b-bd12-1afe32f7b2c0                  | × |

BODY

body de la petición (JSON, form data, etc.)

Figura 37: Repeater con petición SQLi cargada — headers completos y cookie de sesión activa

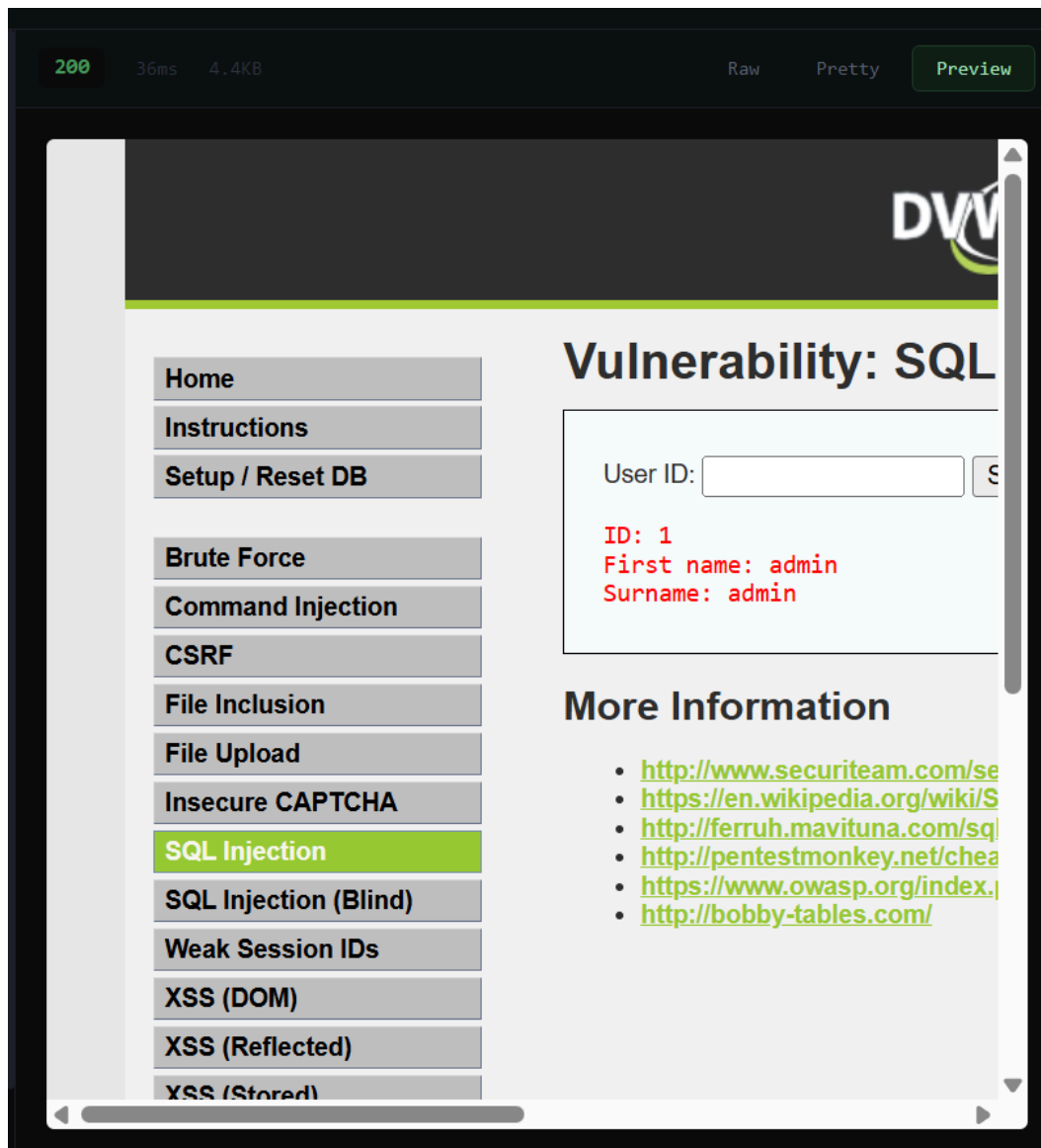


Figura 38: Vista Preview del Repeater — DVWA renderizado con datos reales extraídos por inyección SQL

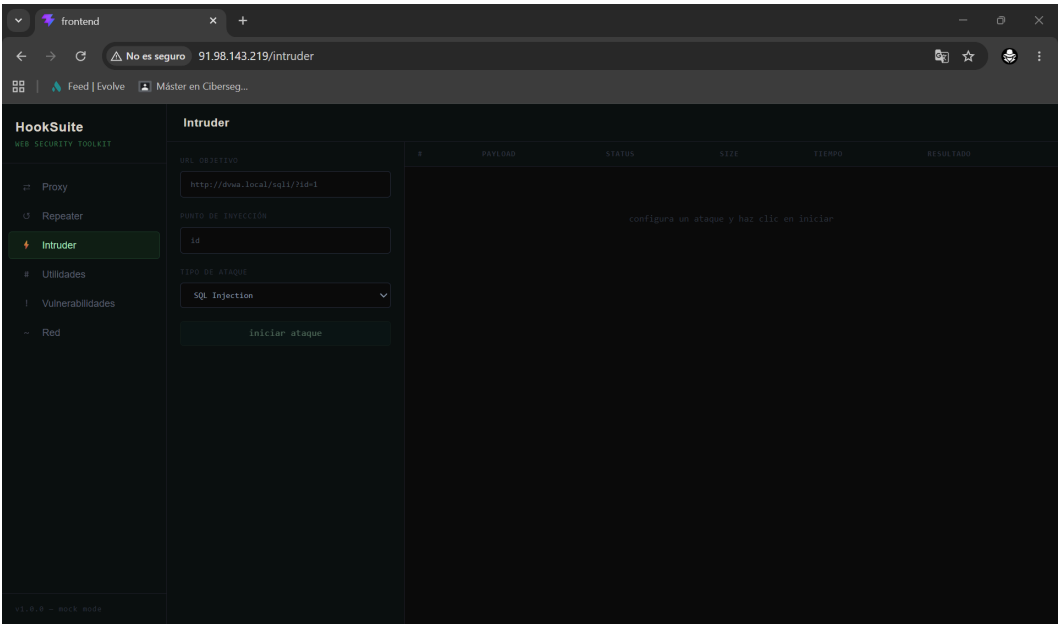


Figura 39: Panel Intruder operativo en producción

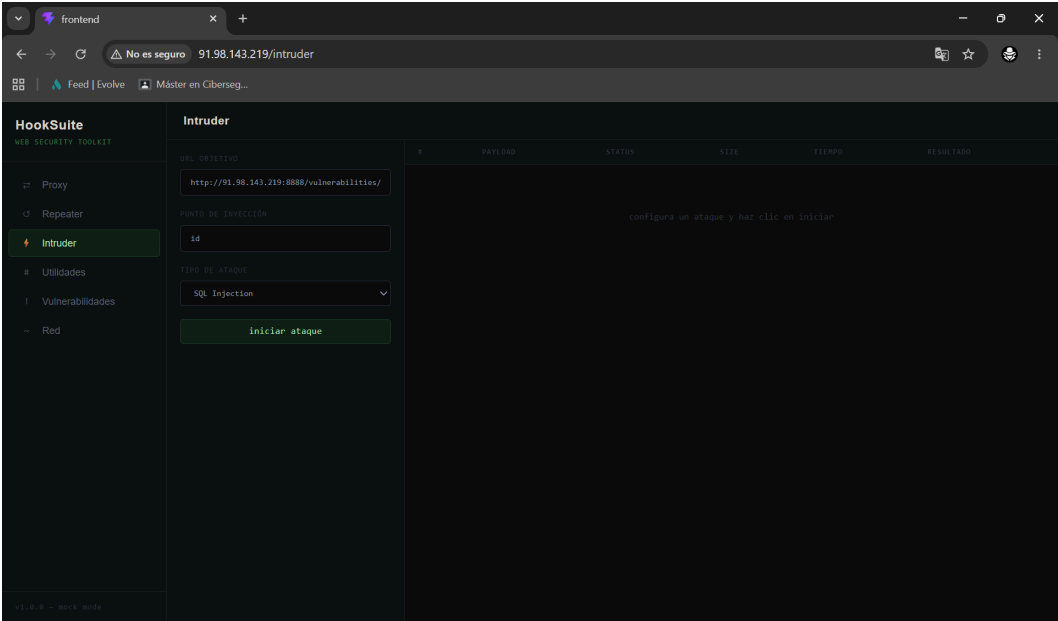


Figura 40: Intruder ejecutando 13 payloads SQLi contra DVWA — resultados en tiempo real

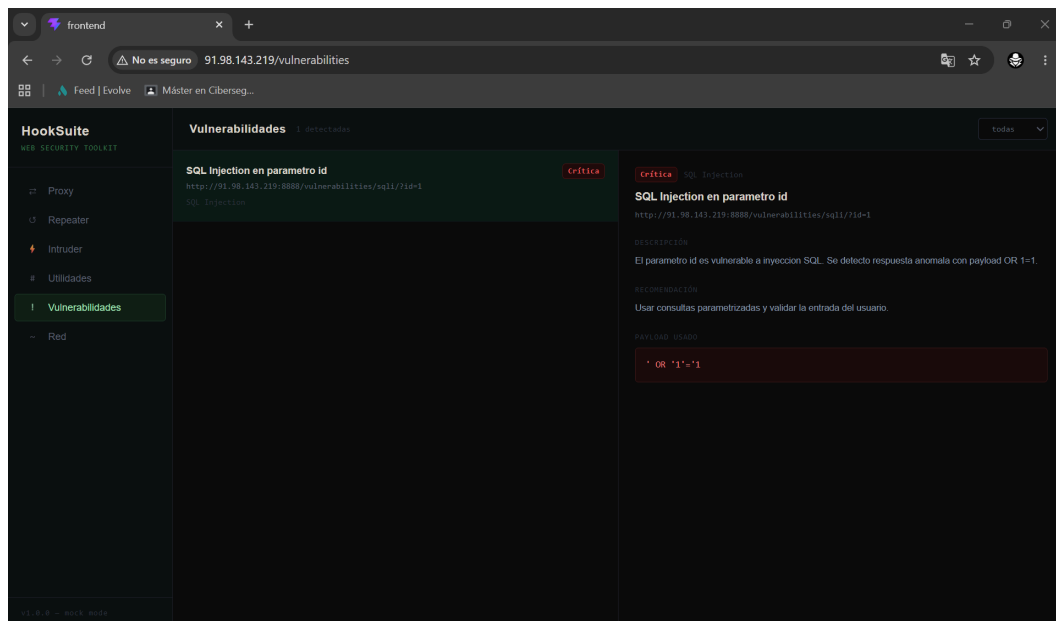


Figura 41: Panel Vulnerabilidades — SQL Injection crítica detectada con payload ' OR '1'='1 y recomendación de mitigación

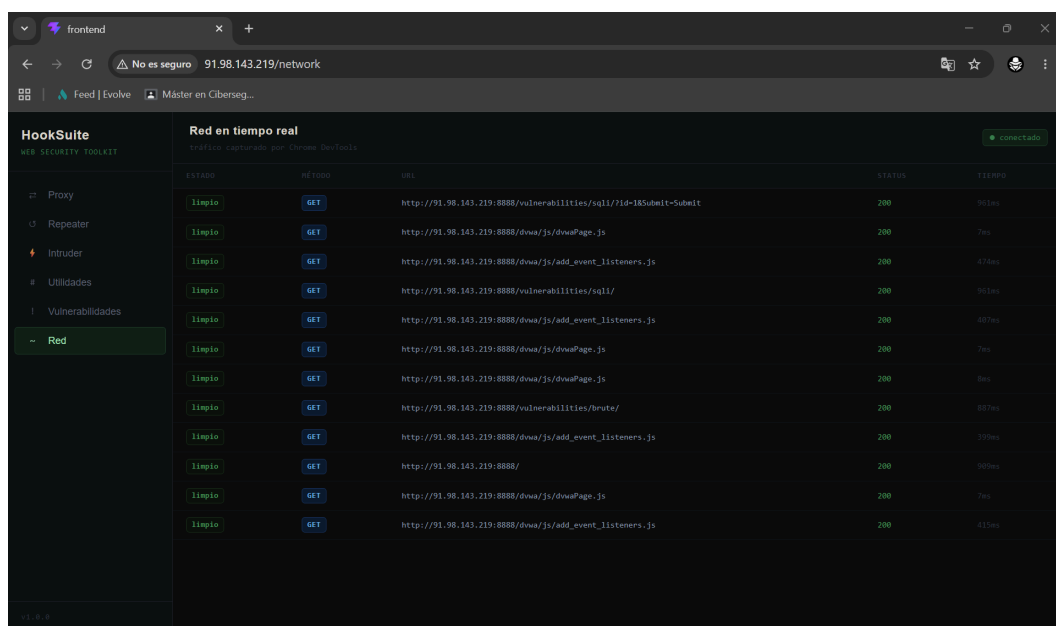


Figura 42: Panel Red de HookSuite recibiendo tráfico capturado por DevTools en el servidor de producción

## El error de los bots

Al abrir el puerto 8080 al exterior para que el proxy TCP fuera accesible desde internet, el servidor recibió inmediatamente tráfico automatizado externo. Los bots entraron de forma descontrolada a través del proxy, generando un volumen de peticiones que el servidor no estaba dimensionado para absorber. El resultado fue la saturación completa del servicio — el proxy dejó de responder y el sistema quedó inutilizable como herramienta de auditoría.

El problema era estructural: un proxy TCP abierto en internet sin autenticación a nivel de conexión es un recurso que cualquier bot puede explotar. Cerrar el puerto resolvía la saturación pero eliminaba la funcionalidad central de la herramienta.

### Intento de solución

Antes de descartar la arquitectura, Carlos trabajó en un intento de salvarla. La propuesta fue convertir el proxy en un servicio por usuario con aislamiento completo: cada login generaría un UID único que arrancaría una instancia de mitmproxy en un puerto aleatorio del rango 10000–60000, abriría ese puerto en el firewall exclusivamente para la IP del usuario mediante un firewall agent desplegado como servicio del sistema, y lo cerraría automáticamente al hacer logout o tras cuatro horas de inactividad. El tráfico interceptado se enviaría a Redis en lugar de directamente al WebSocket — desacoplando la captura del envío y añadiendo resiliencia al sistema — y el Backend consumiría Redis para emitirlo al Frontend.

La solución era técnicamente sólida pero no resolvió el problema en el tiempo disponible. La gestión dinámica del firewall introducía complejidad operacional que se sumaba a la ya existente en la coordinación entre módulos, y el tiempo necesario para estabilizarla comprometía el resto de la entrega.

### Cambio de planteamiento

Fue en una sesión con el profesor donde se definió el camino definitivo. El profesor detectó el problema, planteó varias alternativas y el equipo eligió la que resolvía la saturación de raíz: eliminar el proxy TCP del flujo de auditoría y hacer que el servidor realizara las peticiones HTTP directamente por el auditor. Sin proxy abierto al exterior, sin superficie de ataque para los bots.

Con el nuevo planteamiento definido, el equipo tomó también la decisión de concentrar el tiempo restante hasta la entrega en estabilizar el núcleo de la herramienta — Frontend y Backend. Los módulos de Playwright, DevTools e IA, que habían funcionado en local y estaban desplegados en el servidor en modo polling, quedarían en segundo plano. Su activación completa pasaría a ser el objetivo principal de la Práctica 2.

### Nueva arquitectura

Con la decisión tomada, Jose María reconvirtió el sistema a la nueva arquitectura. Eliminó el modelo de proxy interceptor y lo sustituyó por el nuevo planteamiento: el servidor audita directamente por el auditor, realizando las peticiones HTTP de forma activa en lugar de interceptar el tráfico pasivamente. La pieza técnica central fue el cliente httpx persistente por sesión — un AsyncClient compartido por todos los módulos bajo el mismo token que acumula automáticamente las cookies de la sesión de auditoría. Esta decisión resolvió de raíz el problema de la autenticación compartida: el auditor hace login desde el Repeater y el Spider y el Intruder heredan automáticamente esa sesión sin configuración adicional. Sobre esa base completó el spider httpx, el motor de fuzzing del Intruder y los parsers de raw HTTP y cURL del Repeater. mitmproxy se mantuvo en el código sin eliminarlo, con vistas a su posible uso en auditorías de aplicaciones móviles en la Práctica 2.

Macarena realizó en paralelo una mejora visual del Frontend — navegación horizontal en topbar en lugar del sidebar vertical, sistema de variables CSS con modo claro y modo oscuro, y nuevos componentes de UI con paneles redimensionables. El nuevo diseño se desplegó en el servidor manteniendo toda la lógica funcional intacta.

Al cierre de esta fase el núcleo de HookSuite estaba operativo con la nueva arquitectura: el panel Proxy mostraba las peticiones que el servidor realizaba por el auditor en tiempo real, el Repeater reenviaba peticiones con gestión automática de cookies, el Intruder ejecutaba ataques con paralelismo controlado y el panel de Vulnerabilidades recibía alertas por WebSocket. El flujo completo de auditoría — spider sin autenticación, login desde el Repeater, spider autenticado, Intruder detectando SQLi — fue validado end-to-end contra DVWA antes de la entrega.

## Estado actual de la herramienta

### Frontend

El Frontend es la interfaz visual de HookSuite — un dashboard web accesible desde cualquier navegador sin instalación adicional. Construido sobre React 19, Vite y Tailwind CSS, con JetBrains Mono como tipografía de código, se comunica con el Backend exclusivamente mediante WebSocket para recibir eventos en tiempo real y REST para enviar las acciones del auditor. La gestión del estado compartido entre paneles se centraliza en un contexto global que mantiene la conexión WebSocket activa, el identificador de sesión UUID del auditor y los datos que fluyen entre los distintos módulos de la interfaz. El diseño incorpora un sistema de variables CSS con soporte de modo oscuro y modo claro, navegación horizontal en topbar y paneles redimensionables.

El Frontend se organiza en seis paneles accesibles desde el topbar:

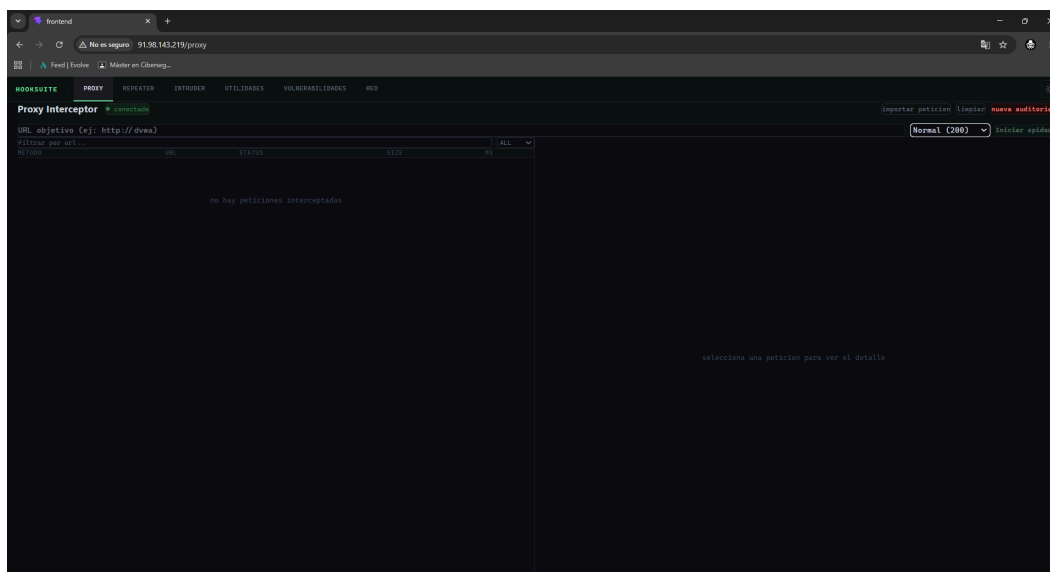


Figura 43: Panel Proxy con el nuevo diseño — topbar horizontal, WebSocket conectado y spider listo para lanzar

El **panel Proxy** es el centro de operaciones de la auditoría. El auditor introduce la URL objetivo, selecciona la velocidad del análisis — rápido, normal o completo, que determina el número máximo de páginas que el spider visitará — y lanza el proceso. Las peticiones que el servidor realiza por el auditor aparecen en tiempo real agrupadas por URL. Los formularios detectados en cada página se muestran como subelementos desplegables bajo su URL correspondiente, lo que permite identificar de un vistazo los vectores de ataque disponibles. El panel de estado muestra las cookies de sesión activas en verde cuando el auditor está autenticado, y ofrece tres acciones: liberar la sesión activa sin perder el historial de peticiones, limpiar el panel manteniendo la sesión, o iniciar una nueva auditoría completa reseteando todo el estado.

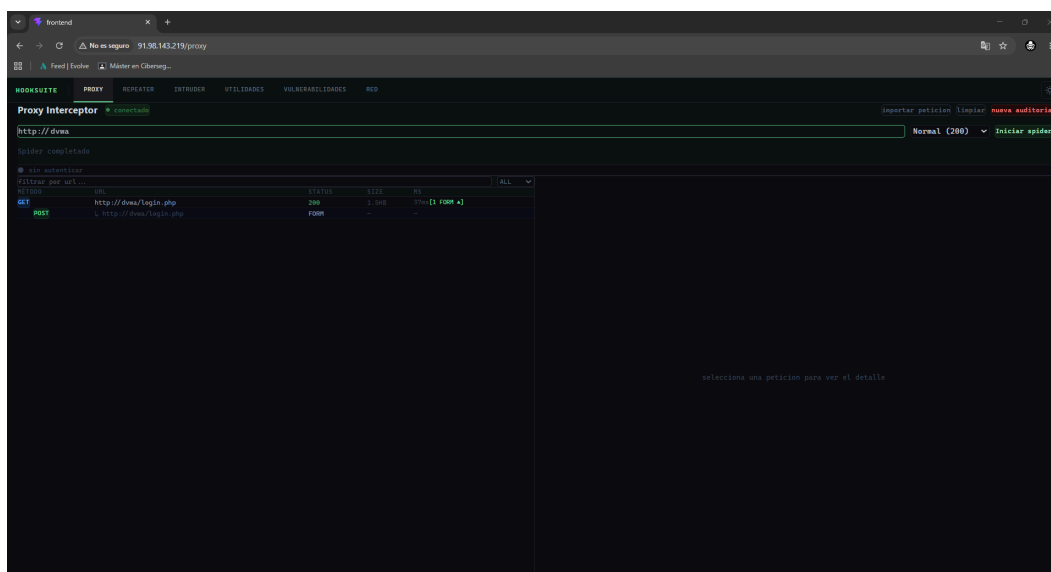


Figura 44: Panel Proxy interceptando peticiones en tiempo real — petición de login de DVWA capturada con formulario detectado

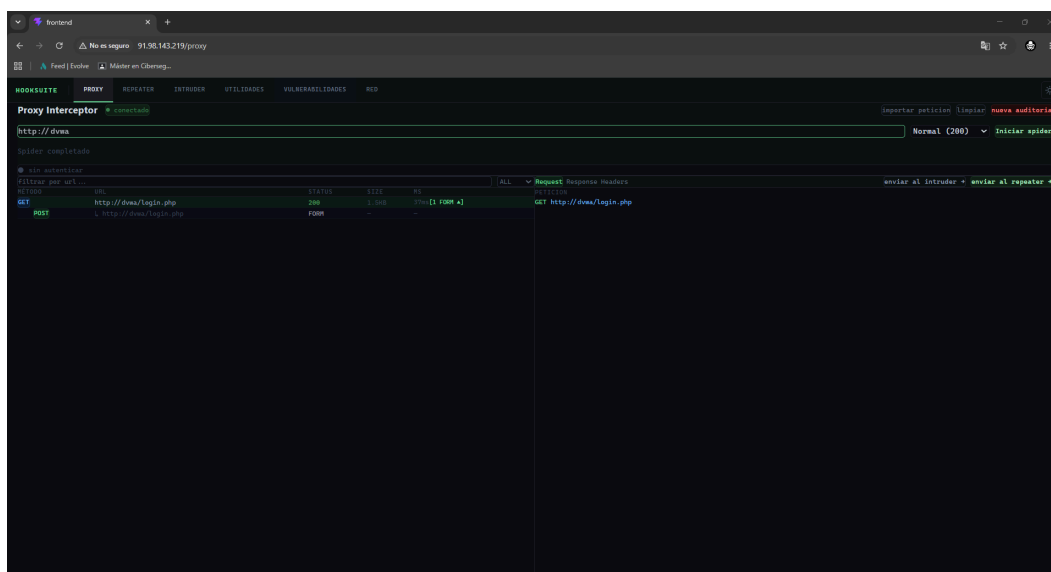


Figura 45: Formulario POST desplegado en el panel Proxy — botones de envío al Repeater y al Intruder visibles

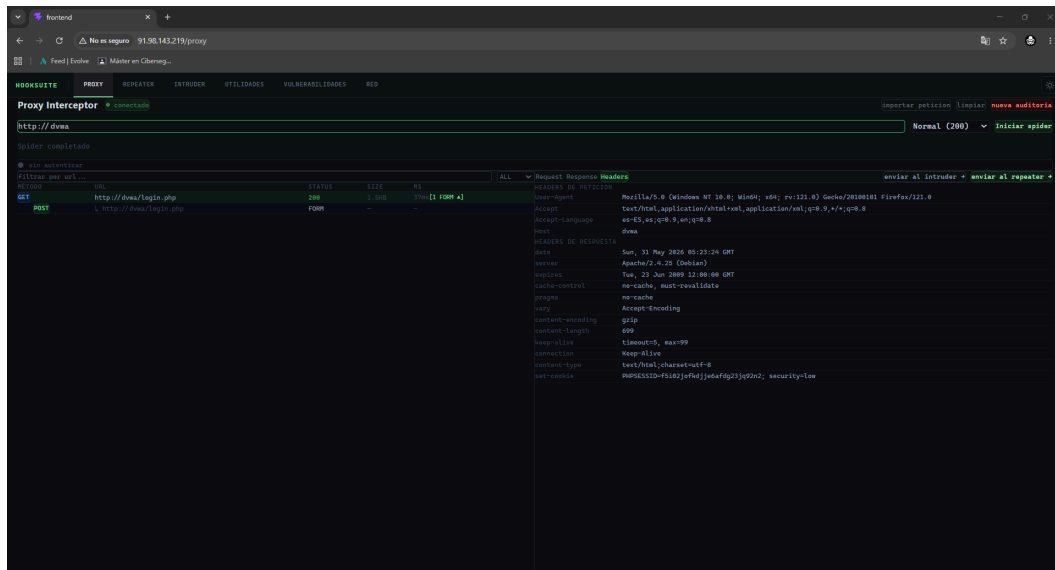


Figura 46: Pestaña Headers del detalle de petición — headers de petición y respuesta con cookie PHPSESSID visible

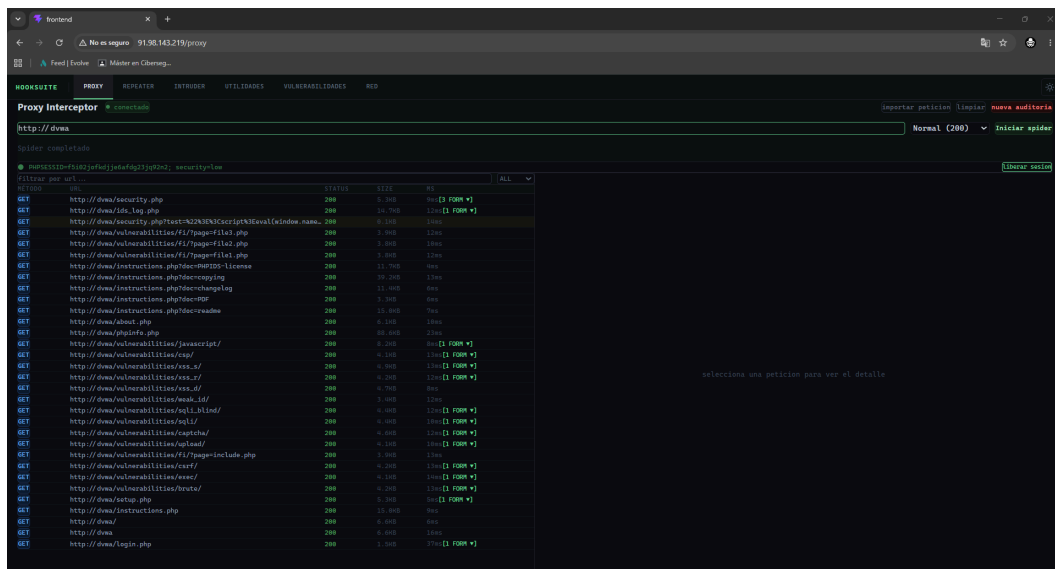


Figura 47: Panel Proxy con sesión autenticada — spider navegando DVWA con todas las páginas descubiertas y formularios detectados

El **Repeater** permite modificar y reenviar cualquier petición manualmente. El auditor puede enviar al Repeater cualquier petición interceptada en el panel Proxy con un solo clic, y desde ahí modificar el método HTTP, la URL, los headers y el body antes de reenviarla. La respuesta se muestra en cuatro vistas: Raw muestra la respuesta tal como llega del servidor; Pretty formatea automáticamente el JSON para facilitar su lectura; Preview renderiza el HTML de la respuesta en un iframe reescribiendo las URLs relativas para que los recursos del objetivo se carguen correctamente; y Headers muestra los headers de respuesta con las cookies resaltadas en verde para identificarlas fácilmente.

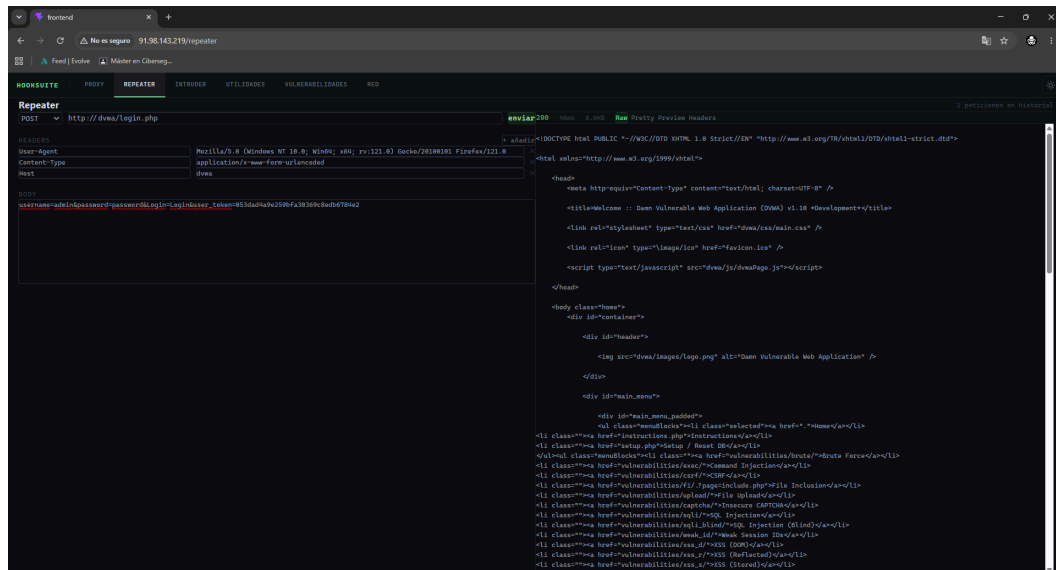


Figura 48: Repeater — vista Raw con la respuesta de login exitoso en DVWA

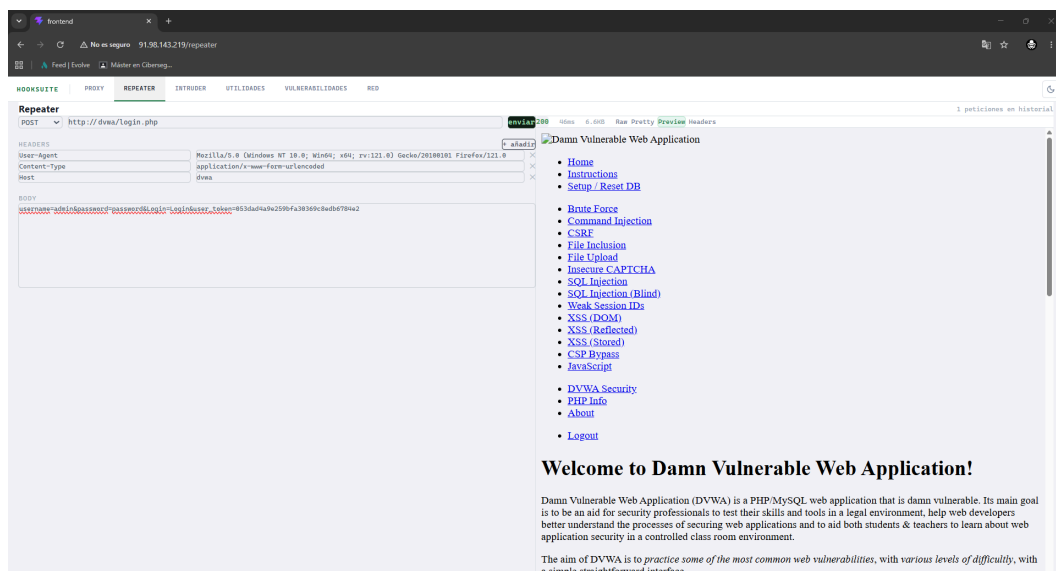


Figura 49: Repeater — vista Preview con DVWA renderizado tras autenticación exitosa

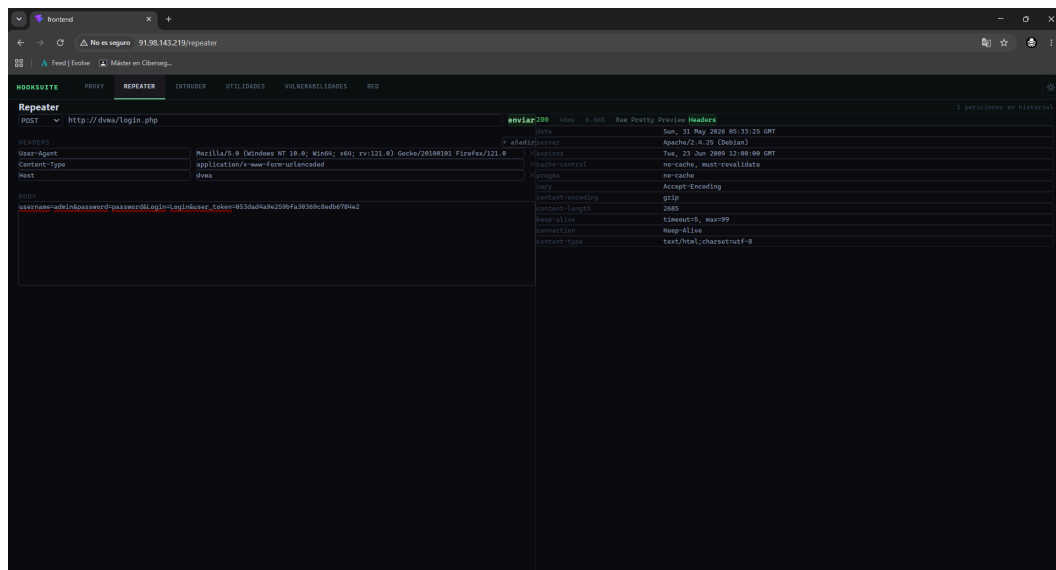


Figura 50: Repeater — vista Headers con los headers de respuesta del servidor

El **Intruder** automatiza el fuzzing de parámetros al estilo Burp Suite. Al recibir una petición detecta automáticamente todos los parámetros GET y POST presentes. El auditor selecciona el parámetro que quiere atacar marcándolo con el símbolo \* — el marcado se resalta en naranja en tiempo real tanto en la URL como en el body. El sistema sustituye ese marcador por cada payload de la lista seleccionada y envía todas las peticiones de forma automatizada. Soporta cuatro tipos de ataque con sus respectivas listas de payloads: SQL Injection, Blind SQLi, XSS y fuzzing genérico. Los resultados se muestran en una tabla en tiempo real donde las peticiones que reciben una respuesta identificada como vulnerable se marcan en rojo.

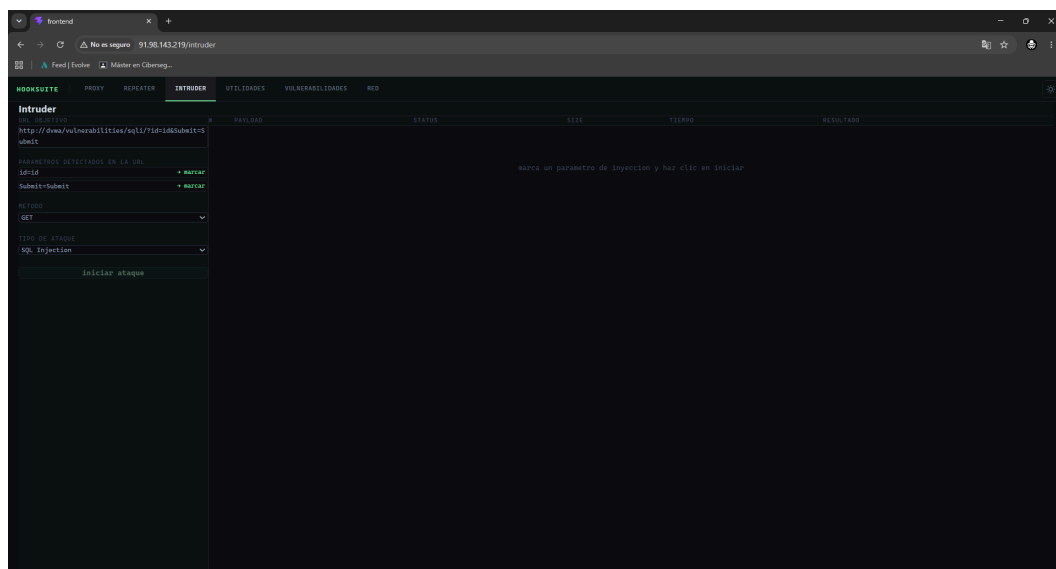


Figura 51: Intruder — parámetro id marcado con \* y ataque SQL Injection configurado

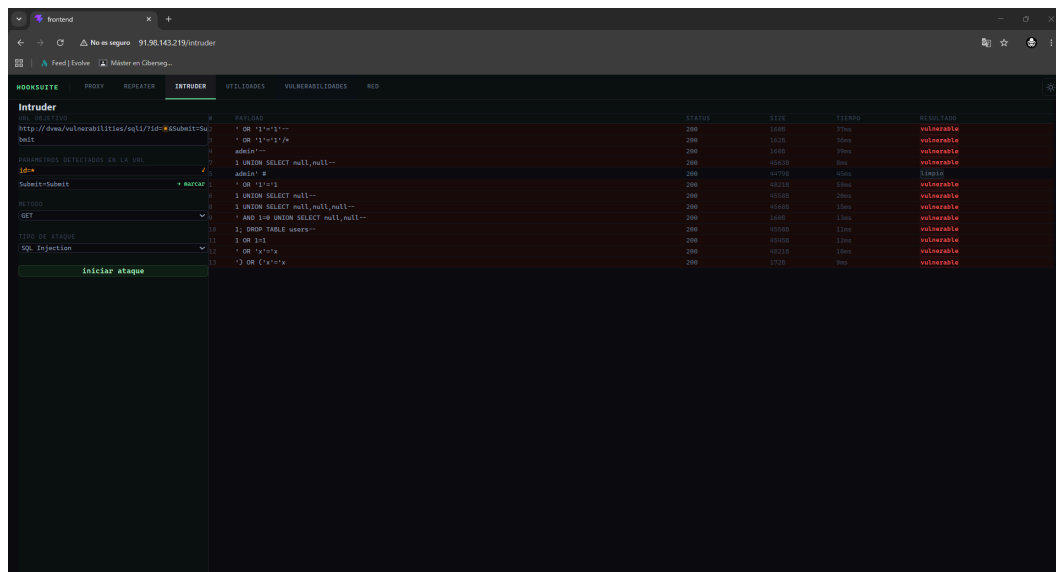


Figura 52: Intruder — 13 payloads ejecutados contra DVWA SQLi, 12 vulnerables detectados en tiempo real

Las **Utilidades** agrupan cuatro herramientas auxiliares de uso frecuente en auditorías web. El Encoder/Decoder transforma texto entre los formatos más comunes — Base64, URL encoding, HTML encoding y decodificación de tokens JWT. El Hash Generator calcula los hashes MD5, SHA1, SHA256 y SHA512 de cualquier texto. El Regex Tester permite probar expresiones regulares contra texto de prueba con resaltado visual de los matches en tiempo real. El Payload Generator organiza colecciones de payloads por tipo de ataque — SQLi, Blind SQLi, XSS y fuzzing genérico — con opción de copiar payloads individuales o la lista completa.

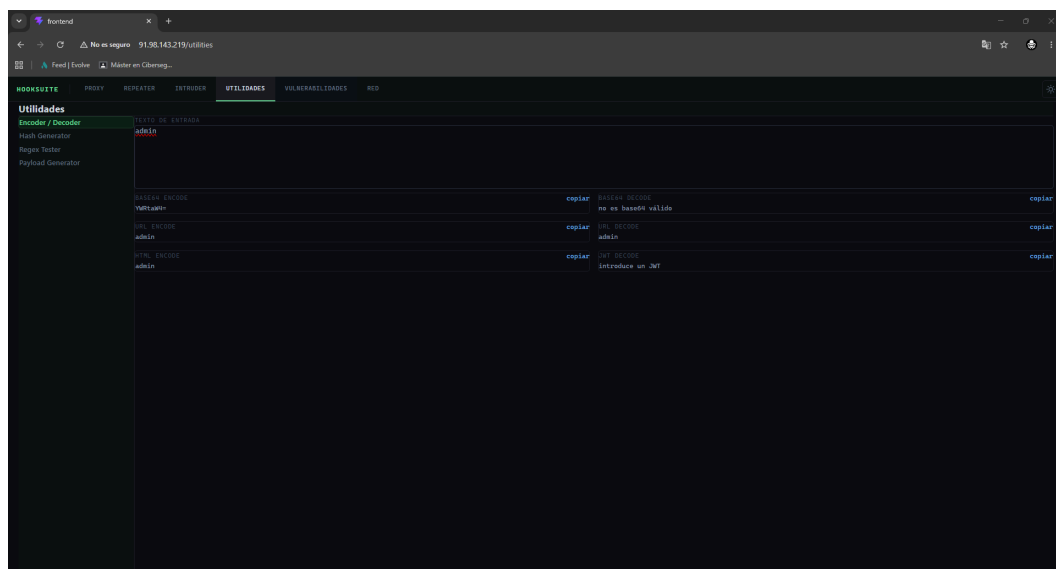


Figura 53: Utilidades — Encoder/Decoder con texto 'admin' transformado a Base64, URL encoding y HTML encoding

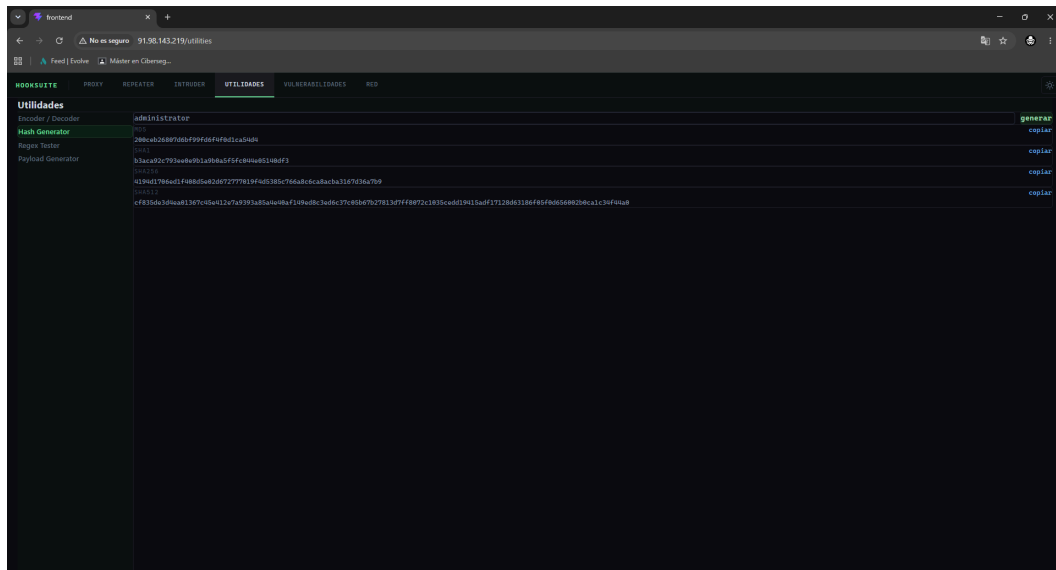


Figura 54: Utilidades — Hash Generator con hashes MD5, SHA1, SHA256 y SHA512 de 'administrator'

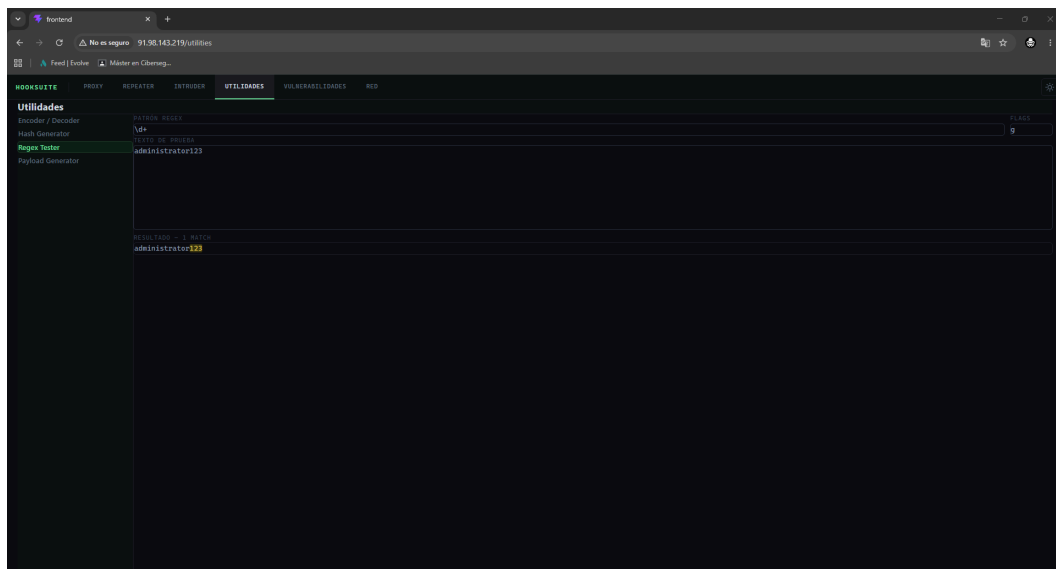


Figura 55: Utilidades — Regex Tester con patrón \d+ y match resaltado en 'administrator123'

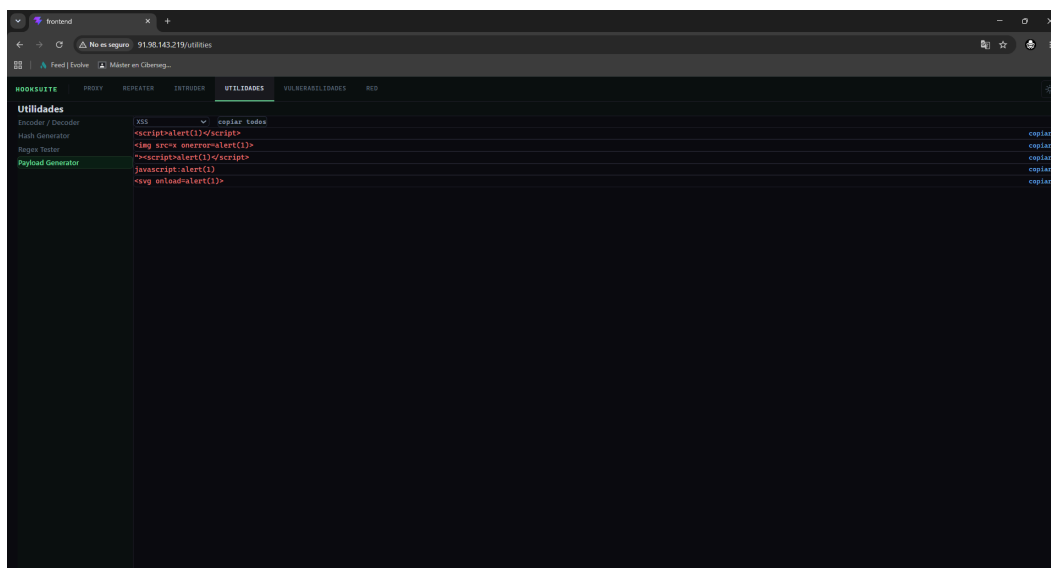


Figura 56: Utilidades — Payload Generator con lista de payloads XSS listos para copiar

El **panel Vulnerabilidades** y el **panel Red** están completamente contruidos e integrados en la interfaz. El panel Vulnerabilidades está diseñado para recibir las detecciones del módulo de IA clasificadas por severidad — crítica, alta, media y baja — con descripción de la vulnerabilidad, payload utilizado y recomendación de mitigación. El panel Red está diseñado para mostrar el tráfico capturado por el módulo DevTools en tiempo real, con código de colores para identificar peticiones limpias, sospechosas y vulnerables. Ambos paneles permanecen inactivos en esta entrega porque los módulos que los alimentan están pendientes de integración completa en la Práctica 2.

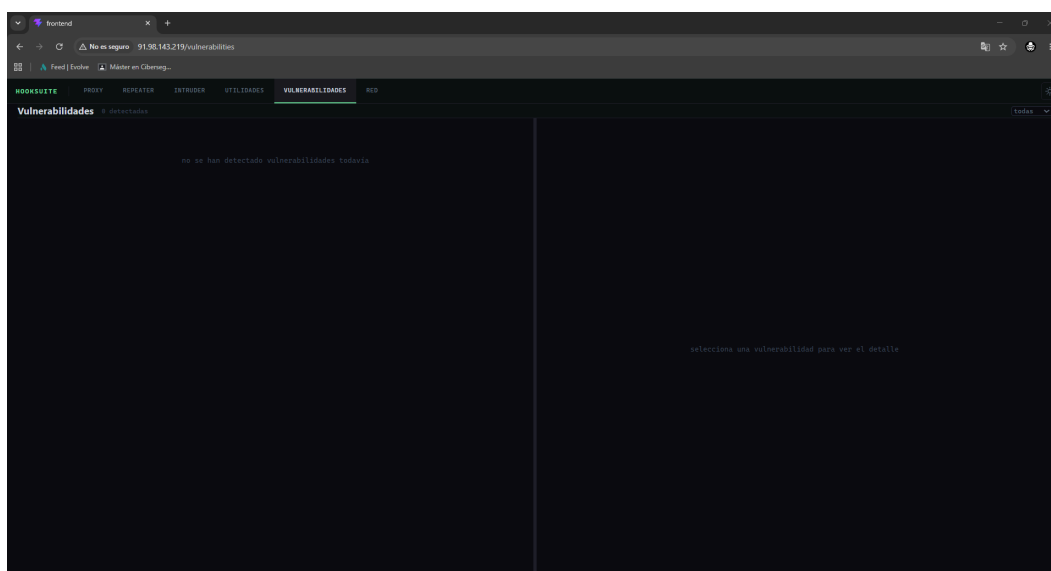


Figura 57: Panel Vulnerabilidades — construido e integrado, pendiente de activación con el módulo IA en la Práctica 2

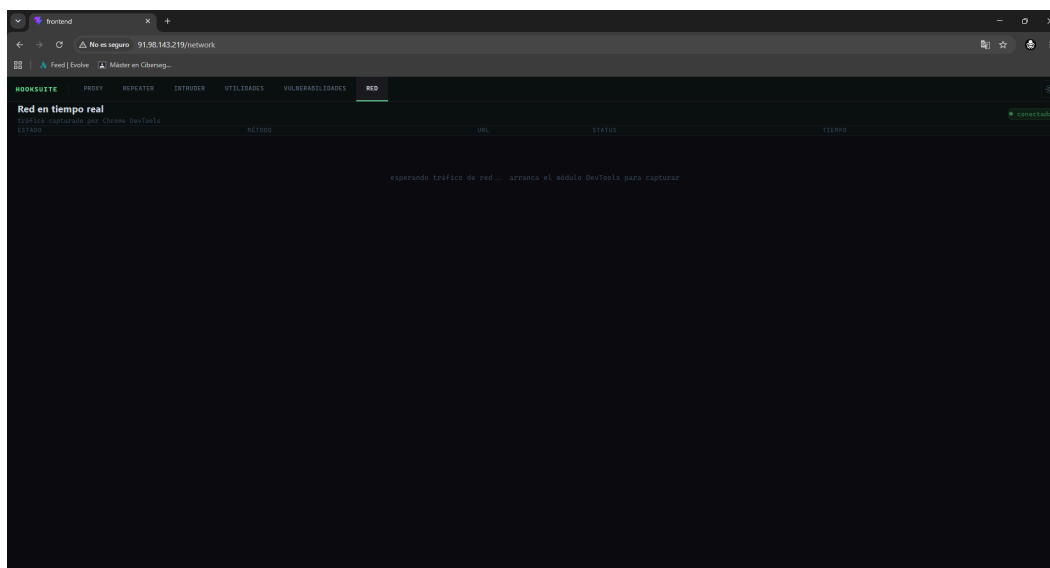


Figura 58: Panel Red — construido e integrado, pendiente de activación con el módulo DevTools en la Práctica 2

## Backend

El Backend es el núcleo del sistema — el único módulo que habla con todos los demás y el que hace posible que HookSuite funcione como una herramienta de auditoría real. Construido sobre Python 3.11 y FastAPI, gestiona las sesiones de auditoría, ejecuta todas las peticiones HTTP por el auditor, emite los resultados al Frontend en tiempo real mediante WebSockets y expone la API REST que coordina el resto de módulos.

El módulo se organiza en seis bloques funcionales:

El **servidor FastAPI** es el punto de entrada del sistema. Arranca con CORS habilitado para aceptar peticiones desde cualquier origen, registra todos los routers de la aplicación bajo el prefijo `/api/`, e inicia al arranque dos tareas asíncronas en segundo plano: un consumidor Redis y un gestor de limpieza de sesiones expiradas. La documentación interactiva de la API — generada automáticamente por FastAPI — está disponible en `/docs` y lista todos los endpoints con sus modelos de entrada y salida.

El **sistema de sesiones y WebSockets** es la pieza que permite que múltiples auditores trabajen simultáneamente sin interferirse. Cada auditor recibe un token UUID único al conectarse. El `SessionManager` mantiene en memoria el estado completo de cada sesión — historial de peticiones, estado del Intruder, paquetes de red, vulnerabilidades detectadas — y registra la conexión WebSocket asociada a cada token. El canal WebSocket en `/ws/{token}` mantiene la conexión viva mediante pings cada 30 segundos y emite eventos tipados con la estructura `{type, payload}` cada vez que ocurre algo relevante en el servidor. El método `emit_all()` permite difundir eventos a todas las sesiones activas simultáneamente.

El **spider httpx** es el módulo de reconocimiento activo. Dado un punto de entrada y un token de sesión, navega la aplicación objetivo de forma autónoma usando un cliente `httpx` persistente que comparte con el Repeater y el Intruder. Esta persistencia es lo que permite el flujo de auditoría autenticada: el auditor hace login desde el Repeater, y el spider hereda

automáticamente esa sesión y navega autenticado sin configuración adicional. El spider implementa un algoritmo BFS con scope restringido al dominio objetivo, filtra extensiones estáticas irrelevantes y varía los User-Agent de forma aleatoria entre peticiones. Para cada página visitada extrae mediante expresiones regulares todos los formularios HTML — método, acción y campos — y los emite al Frontend como peticiones independientes con status FORM. La velocidad es configurable en tres presets — rápido, normal y completo — que determinan el número máximo de páginas a visitar.

El **cliente httpx persistente por sesión** es la decisión técnica más importante del módulo. En lugar de crear un cliente HTTP nuevo para cada petición, el backend mantiene un diccionario que asocia cada token de sesión con un AsyncClient de httpx configurado con seguimiento de redirects y verificación SSL desactivada. Este cliente acumula automáticamente las cookies que el servidor objetivo va devolviendo a lo largo de la auditoría. El resultado es que el Repeater, el Spider y el Intruder comparten implícitamente la misma sesión HTTP — incluyendo las cookies de autenticación — sin que el auditor tenga que copiar ni gestionar nada manualmente. Cuando el backend detecta nuevas cookies, emite un evento WebSocket al Frontend para mostrarlas en el panel de estado de auditoría.

El **motor de fuzzing del Intruder** ejecuta ataques automatizados contra parámetros específicos. Recibe del Frontend la URL objetivo con el punto de inyección marcado con \*, el tipo de ataque y la configuración de paralelismo. Sustituye el marcador por cada payload de la lista correspondiente y lanza todas las peticiones de forma concurrente usando `asyncio.Semaphore` con un límite de cinco peticiones simultáneas — elegido para no sobrecargar el servidor de producción. Soporta cuatro tipos de ataque con sus respectivas listas de payloads: SQL Injection con trece vectores, Blind SQLi con nueve variantes incluyendo time-based, XSS con diez payloads, y fuzzing genérico con diecisiete entradas que cubren path traversal, null bytes, templates y cadenas extremadamente largas. La detección de vulnerabilidades SQLi se basa en la búsqueda de patrones de confirmación en la respuesta — `First name:`, `Surname:`, errores de base de datos — que indican que el payload ha producido una respuesta anómala. Los resultados se emiten al Frontend en tiempo real a medida que cada payload completa su ejecución.

Las **utilidades** exponen tres endpoints auxiliares implementados con la librería estándar de Python sin dependencias externas. El generador de hashes calcula MD5, SHA1, SHA256 y SHA512 de cualquier texto. El encoder/decoder transforma texto entre Base64, URL encoding y HTML encoding en ambas direcciones. El regex tester compila y ejecuta expresiones regulares con soporte de flags — case insensitive, multiline, dotall — y devuelve todos los matches con sus posiciones.

El **receptor de paquetes de red** y los **endpoints de integración con Playwright e IA** están completamente implementados en el Backend. El receptor de paquetes expone dos endpoints para recibir los paquetes capturados por el módulo DevTools y distribuirlos a las sesiones activas por WebSocket. Los endpoints de instrucciones y resultados de Playwright permiten al módulo de IA enviar órdenes de ataque y recibir los resultados de su ejecución. El endpoint de vulnerabilidades recibe las detecciones del módulo de IA y las emite al panel correspondiente del Frontend. Estos tres bloques permanecen inactivos en esta entrega

porque los módulos que los alimentan están pendientes de integración completa en la Práctica 2.

### **Playwright, DevTools e IA**

Los módulos de Playwright, DevTools e IA están contruidos y validados en sus entornos locales, y desplegados en el servidor como contenedores Docker en modo polling pasivo. El pivote de arquitectura que ocurrió a mitad del desarrollo redefinió el rol de los tres módulos y el tiempo disponible se concentró en estabilizar el núcleo operativo de la herramienta. Los tres módulos, sus componentes y su lógica interna están descritos en detalle en el apartado de Proceso individual de esta sección. Su integración completa con el Frontend y el Backend es el objetivo principal de la Práctica 2.

SECCIÓN 5

# Guía de despliegue

## Entorno de producción

HookSuite está desplegado en un servidor Hetzner Cloud CX22 con las siguientes especificaciones:

| Parámetro         | Valor               |
|-------------------|---------------------|
| Proveedor         | Hetzner Cloud       |
| Plan              | CX22                |
| vCPUs             | 2 vCPUs compartidas |
| RAM               | 4 GB                |
| Almacenamiento    | 40 GB NVMe SSD      |
| Tráfico incluido  | 20 TB/mes           |
| Sistema operativo | Ubuntu 24.04 LTS    |
| Kernel            | 6.8.0-117-generic   |
| IP pública        | www.hooksuite.de    |
| Docker            | 29.5.0              |
| Docker Compose    | v5.1.3              |

## Arquitectura de contenedores

El sistema se compone de siete contenedores Docker orquestados con Docker Compose y comunicados a través de la red interna hooksuite-net de tipo bridge:

| Servicio | Imagen base                   | Puerto externo | Estado |
|----------|-------------------------------|----------------|--------|
| nginx    | nginx:alpine                  | 80             | Activo |
| frontend | node:20-alpine + nginx:alpine | —              | Activo |
| backend  | python:3.11-slim              | 8000           | Activo |
| redis    | redis:7-alpine                | —              | Activo |

|            |                      |   |            |
|------------|----------------------|---|------------|
| dvwa       | vulnerables/web-dvwa | — | Activo     |
| playwright | imagen propia        | — | Práctica 2 |
| ia         | imagen propia        | — | Práctica 2 |

## Construcción de los contenedores

### Backend

El contenedor de backend usa Python 3.11-slim como imagen base. Durante la construcción instala `redis-tools`, `iptables` y `build-essential` — necesarios para la gestión dinámica del firewall por sesión. Expone los puertos 8000 (API) y 8080 (mitmproxy). Arranca con `NET_ADMIN` para poder manipular reglas de iptables, y monta el socket del agente de firewall desde el host.

El `entrypoint.sh` implementa el siguiente flujo de arranque:

```
# 1. Espera a que Redis esté disponible
until redis-cli -h redis -p 6379 ping; do sleep 1; done

# 2. Levanta mitmproxy en segundo plano en el puerto 8080
mitmdump --listen-host 0.0.0.0 --listen-port 8080 \
  --set block_global=false \
  --proxyauth hooksuite:audit2026 \
  -s /app/services/mitm_addon.py &

# 3. Arranca el servidor FastAPI en el puerto 8000
uvicorn main:app --host 0.0.0.0 --port 8000
```

mitmproxy está presente en la infraestructura aunque no forma parte del flujo de auditoría principal en esta entrega — el sistema opera con httpx directo. Está disponible para un posible uso futuro sin necesidad de cambios en la infraestructura.

### Frontend

El contenedor de frontend usa un proceso de construcción multietapa. En la primera etapa, Node 20 Alpine compila la aplicación React con Vite inyectando la URL del backend como variable de entorno en tiempo de compilación:

```
ARG VITE_API_URL=http://www.hooksuite.de:8000
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build
```

En la segunda etapa, Nginx Alpine sirve los ficheros estáticos compilados. El resultado es una imagen final ligera sin dependencias de Node en producción.

### Módulos Playwright e IA

Ambos contenedores están desplegados en el servidor en modo polling pasivo — operativos como infraestructura pero sin integración activa con el núcleo del sistema. Su activación completa está planificada para la Práctica 2.

## Enrutamiento Nginx

Nginx actúa como punto de entrada único en el puerto 80 y distribuye el tráfico según la ruta:

| Ruta       | Destino y notas                                           |
|------------|-----------------------------------------------------------|
| /api/      | Backend FastAPI — endpoints REST con cabeceras de proxy   |
| /ws/       | Backend WebSockets — upgrade HTTP/1.1 a WS                |
| /proxy.pac | Backend — fichero PAC de configuración del proxy          |
| /check/    | Backend — endpoint de verificación del proxy activo       |
| /dvwa/     | DVWA — protegido con autenticación básica Nginx           |
| /          | Frontend React — protegido con autenticación básica Nginx |

El acceso al dashboard y a DVWA requiere autenticación HTTP básica gestionada por Nginx mediante fichero htpasswd. El puerto 8080 de mitmproxy no está expuesto al exterior.

## Despliegue desde cero

### Requisitos previos

- Servidor Linux con Ubuntu 24.04 LTS
- Docker 24+ y Docker Compose v2+
- Acceso SSH con usuario root o sudo
- Mínimo 2 vCPUs y 4 GB de RAM recomendados

### Pasos

```
# 1. Instalar Docker
curl -fsSL https://get.docker.com | sh

# 2. Clonar el repositorio
git clone https://github.com/Feet-Lovers/Proyecto-Evolve.git
cd Proyecto-Evolve

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con los valores del entorno

# 4. Generar fichero htpasswd para autenticación Nginx
apt-get install -y apache2-utils
```

```
htpasswd -c infra/htpasswd hooksuite

# 5. Construir y arrancar todos los contenedores
docker compose up -d --build

# 6. Verificar estado
docker compose ps
```

## Operación y mantenimiento

### Ver estado de los contenedores

```
docker compose ps
```

### Ver logs de un servicio

```
docker compose logs backend --tail=50
docker compose logs frontend --tail=50
```

### Reiniciar un servicio

```
docker compose restart backend
```

### Actualizar el código y redespargar

```
git pull origin main
docker compose up -d --build backend
# o para redespargar todos los servicios:
docker compose up -d --build
```

### Parar el sistema completo

```
docker compose down
```

### Verificar accesibilidad

```
curl -s -o /dev/null -w "%{http_code}" http://www.hooksuite.de
```

## SECCIÓN 6

# Manual de uso

---

## Acceso a la herramienta

HookSuite es accesible desde cualquier navegador sin instalación adicional. La herramienta está desplegada en <http://www.hooksuite.de>. Al acceder, Nginx solicita autenticación básica — introducir las credenciales proporcionadas para acceder al dashboard.

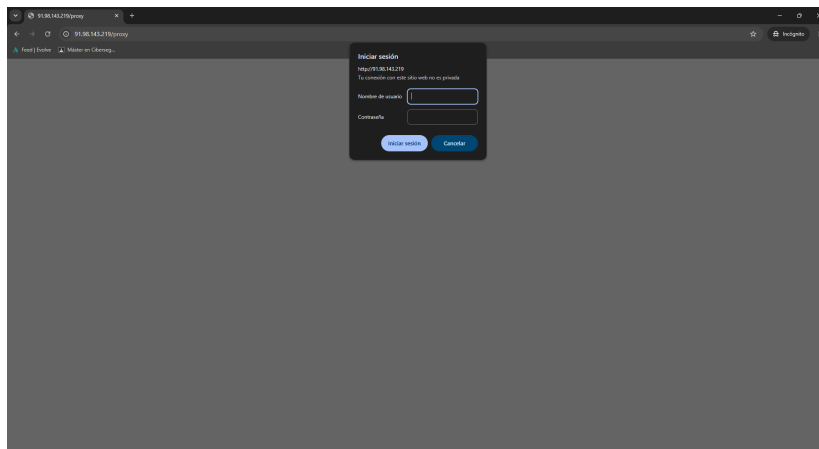


Figura 59: Diálogo de autenticación básica Nginx — credenciales requeridas para acceder al dashboard

Una vez autenticado, el navegador muestra el panel Proxy directamente. La navegación entre los seis paneles se realiza desde el topbar superior. El indicador verde **conectado** en el panel Proxy confirma que el WebSocket con el Backend está activo y la herramienta lista para operar.

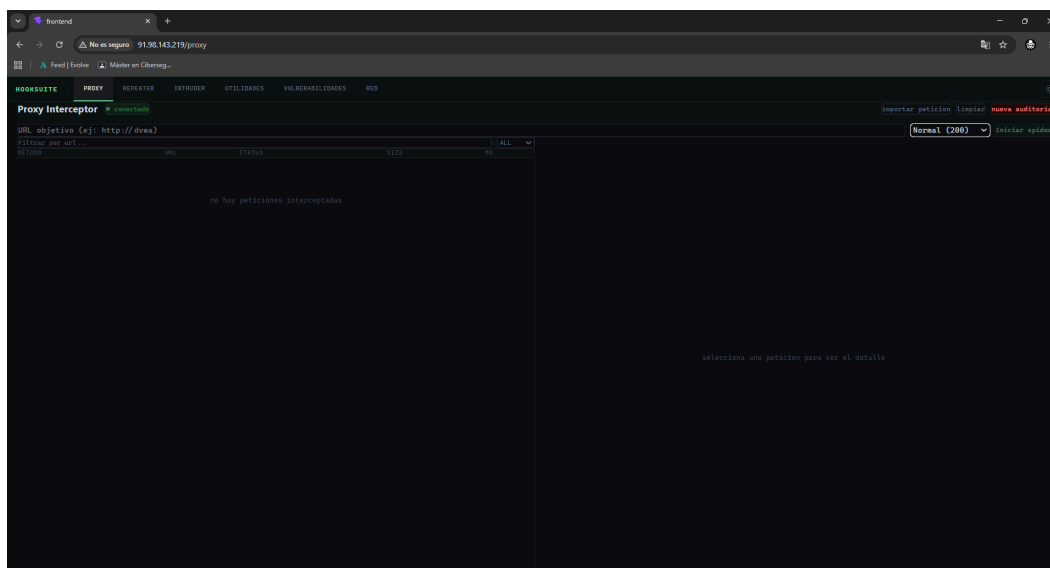


Figura 60: Dashboard de HookSuite — panel Proxy con WebSocket conectado y listo para iniciar una auditoría

## Flujo de auditoría completo

### Paso 1 — Lanzar el spider

En el campo **URL objetivo** del panel Proxy, introducir la URL de la aplicación a auditar. Seleccionar la velocidad de análisis según el alcance deseado — **Rápido** visita hasta 50 páginas, **Normal** hasta 200 y **Completo** hasta 500. Pulsar **Iniciar spider**.

El spider navega la aplicación de forma autónoma realizando peticiones HTTP por el auditor. Cada petición descubierta aparece en tiempo real en la tabla del panel Proxy con su método, URL, status code, tamaño y tiempo de respuesta. Los formularios detectados en cada página aparecen como subelementos desplegados bajo su URL correspondiente, identificados con el indicador **[N FORM ▼]**.

El panel Proxy ofrece tres acciones en la esquina superior derecha:

- **importar petición** — abre un modal para pegar una petición en formato raw HTTP o cURL y enviarla directamente al Repeater sin necesidad de interceptarla con el spider.
- **limpiar** — borra el historial de peticiones del panel manteniendo la sesión activa y las cookies acumuladas.
- **nueva auditoría** — reset completo del sistema: borra el historial, libera las cookies de sesión, resetea la URL activa y reinicia el WebSocket. Equivale a empezar una auditoría desde cero.

Cuando hay una sesión autenticada activa, aparece adicionalmente el botón **liberar sesión** — cierra el cliente httpx y elimina las cookies acumuladas manteniendo el historial visible. Útil para cambiar de usuario sin perder lo interceptado.

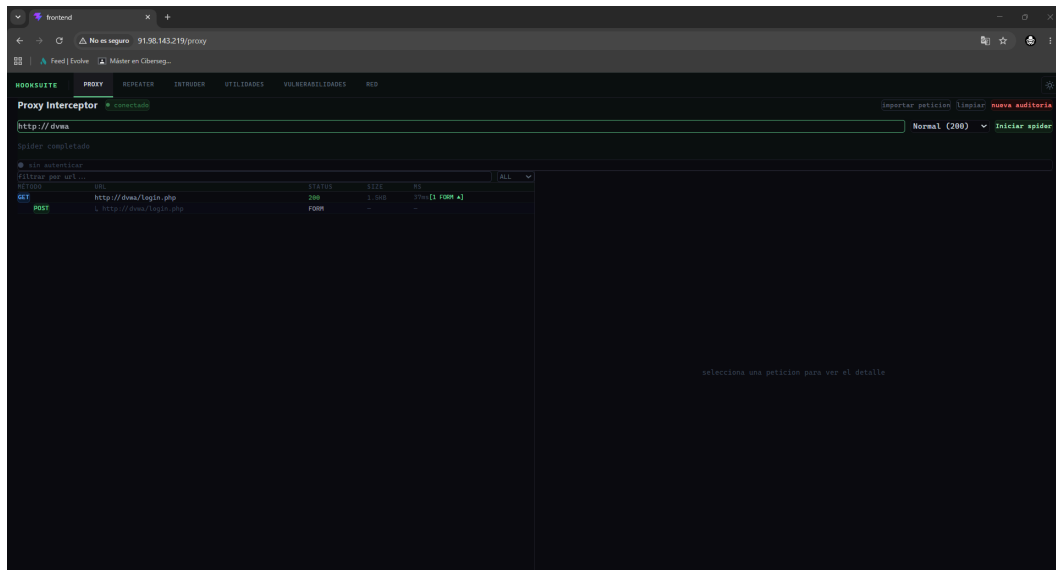


Figura 61: Spider en ejecución — petición de login de DVWA capturada con formulario detectado

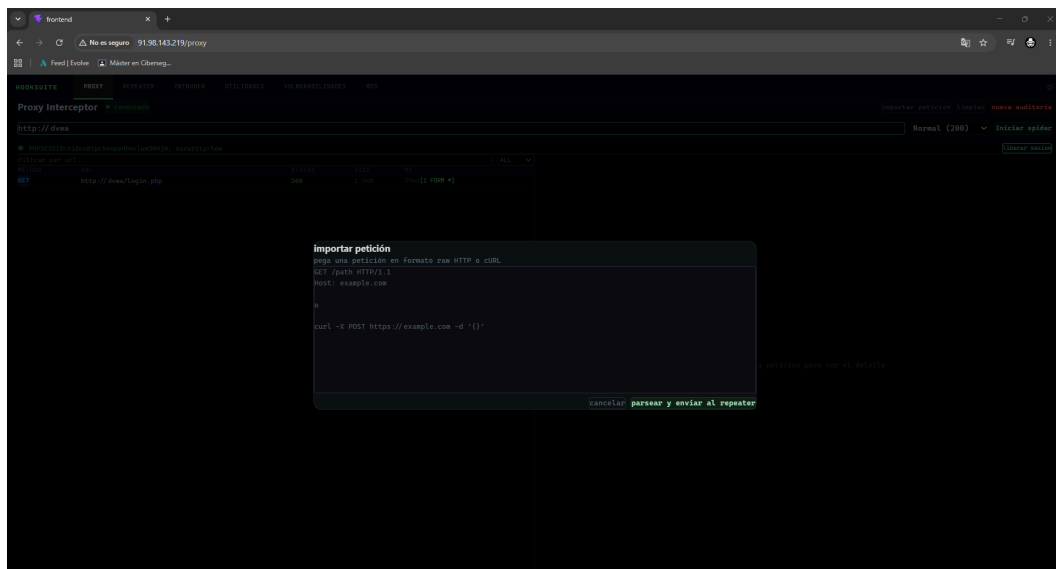


Figura 62: Modal importar petición — editor para pegar raw HTTP o cURL con botón parsear y enviar al repeater

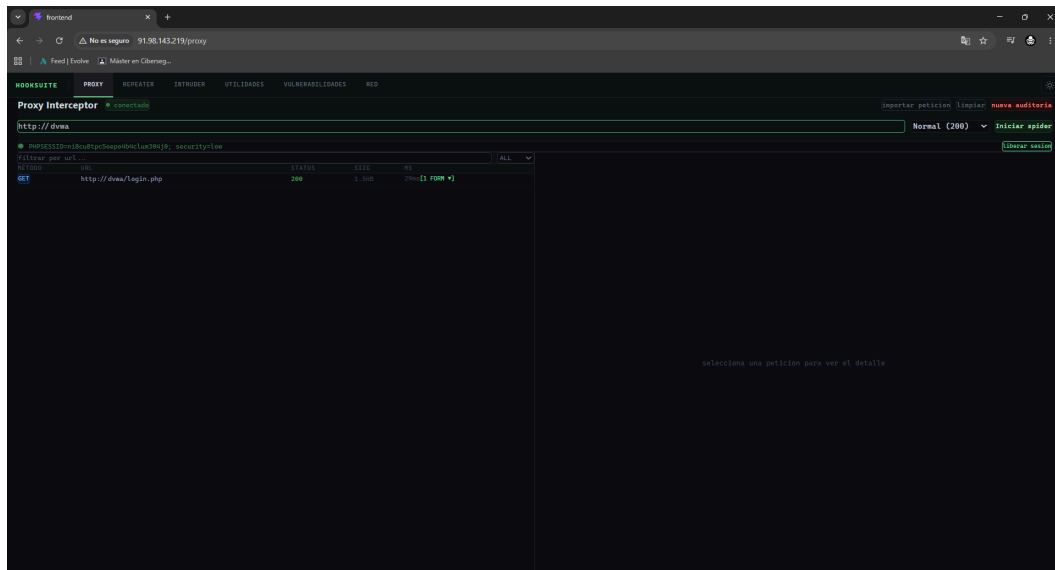


Figura 63: Panel Proxy con sesión autenticada — cookie PHPSESSID activa y botón liberar sesión visible

## Paso 2 — Login desde el Repeater

Cuando el spider detecta el formulario de login de la aplicación objetivo, aparece como subelemento desplegable bajo la URL del login. Desplegar el formulario pulsando **[N FORM ▼]** y seleccionar el formulario POST. Hacer clic en él para ver su detalle en el panel derecho y pulsar **enviar al repeater** →.

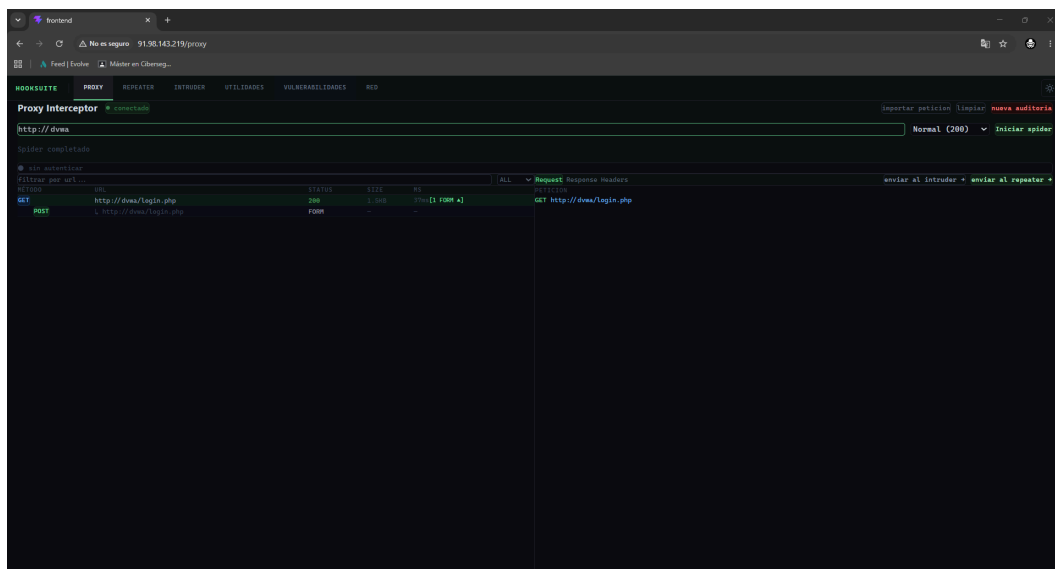


Figura 64: Formulario de login detectado — botones de envío al Repeater y al Intruder visibles

El Repeater recibe la petición con el método POST ya configurado, los headers correctos — incluyendo Content-Type: application/x-www-form-urlencoded — y el body con todos los campos del formulario rellenos con sus valores por defecto, incluido el token CSRF si la aplicación lo requiere.

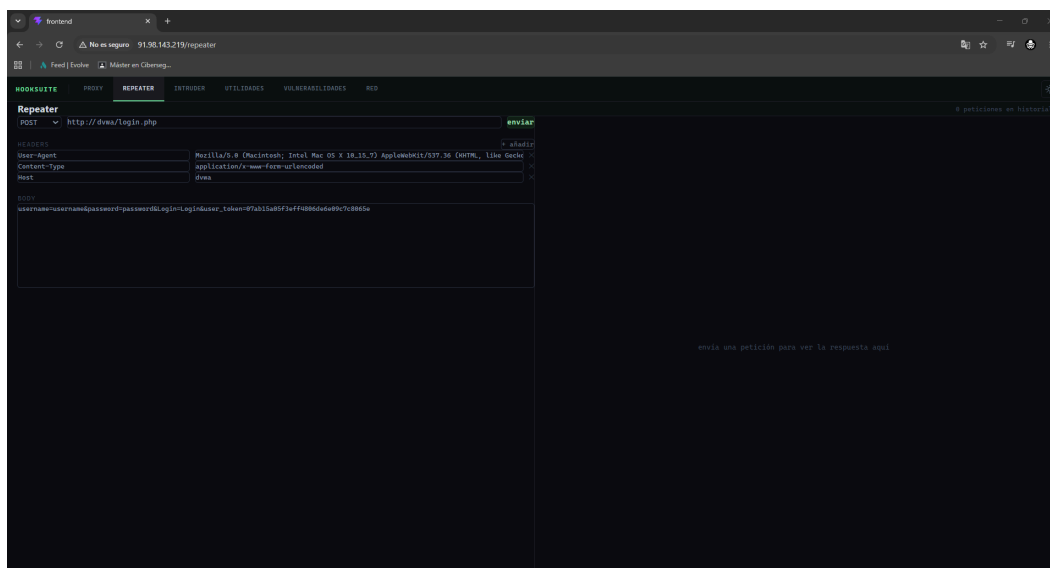


Figura 65: Repeater con el formulario POST recibido — body con campos del formulario y token CSRF incluidos

Modificar los campos de credenciales en el body con los valores correctos y pulsar **enviar**. Si el login es exitoso, la vista Preview mostrará el dashboard de la aplicación autenticada. La cookie de sesión queda automáticamente acumulada en el cliente httpx del Backend — el Spider y el Intruder la heredan sin configuración adicional.

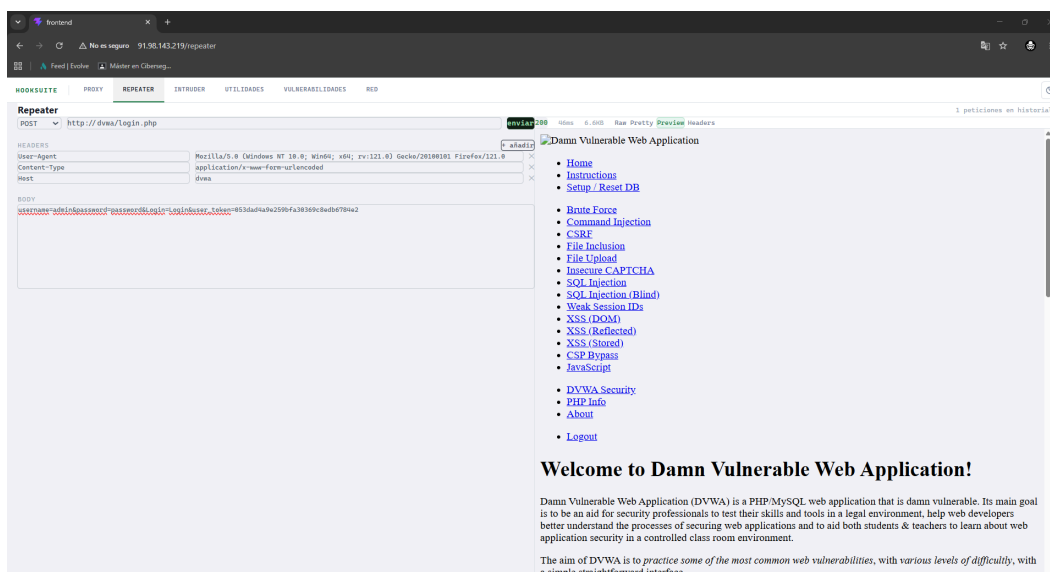


Figura 66: Vista Preview del Repeater — DVWA autenticado tras login exitoso

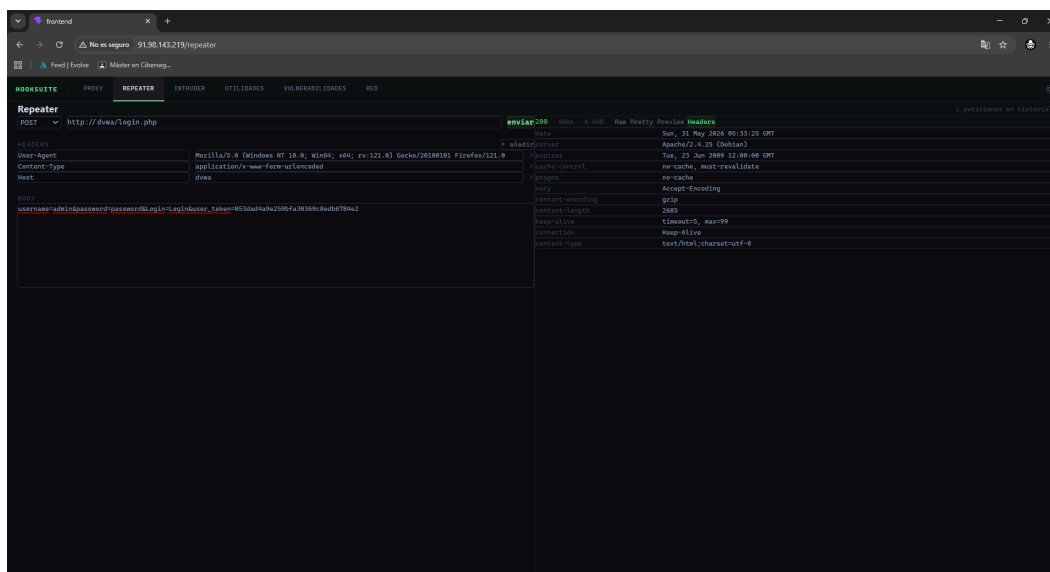


Figura 67: Vista Headers del Repeater — headers de respuesta del servidor

### Paso 3 — Spider autenticado

Con la sesión activa, volver al panel Proxy y lanzar el spider de nuevo. Esta vez navega con las cookies de sesión acumuladas — descubre todas las páginas internas de la aplicación que requieren autenticación. El indicador de cookies en verde confirma que el spider opera autenticado.

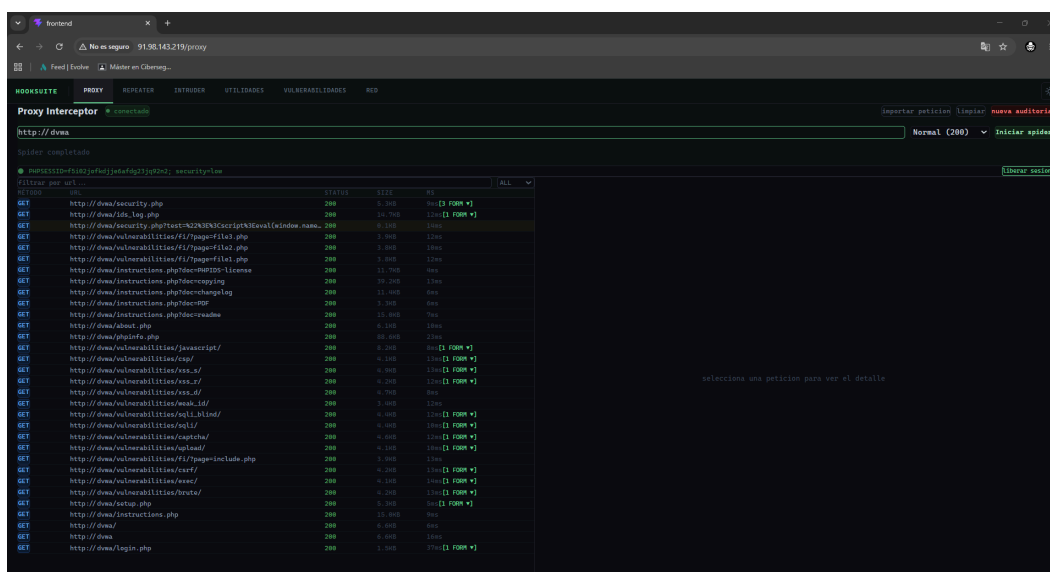


Figura 68: Panel Proxy con sesión autenticada — todas las páginas de DVWA descubiertas con formularios detectados

### Paso 4 — Enviar al Intruder y configurar el ataque

Desplegar el formulario de la página objetivo pulsando **[N FORM ▼]**. Seleccionar el formulario que se quiere atacar y pulsar **enviar al intruder** →. El Intruder recibe automáticamente la URL con los parámetros detectados.

En el Intruder, pulsar → **marcar** junto al parámetro que se quiere atacar — el marcador \* se resalta en naranja en la URL. Seleccionar el tipo de ataque en el desplegable: **SQL Injection**, **Blind SQLi**, **XSS** o **Fuzzing genérico**. Pulsar **iniciar ataque**.

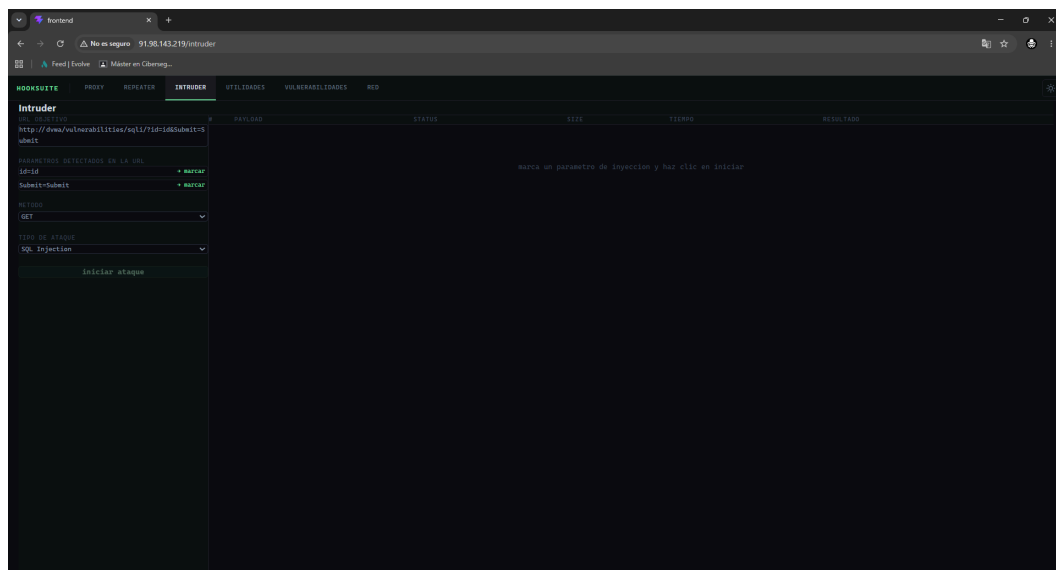


Figura 69: Intruder configurado — parámetro id marcado con \* y tipo de ataque SQL Injection seleccionado

## Paso 5 — Resultados del ataque

El Intruder ejecuta todos los payloads de forma concurrente y muestra los resultados en tiempo real. Las peticiones que producen una respuesta identificada como vulnerable se marcan en rojo en la columna **Resultado**.

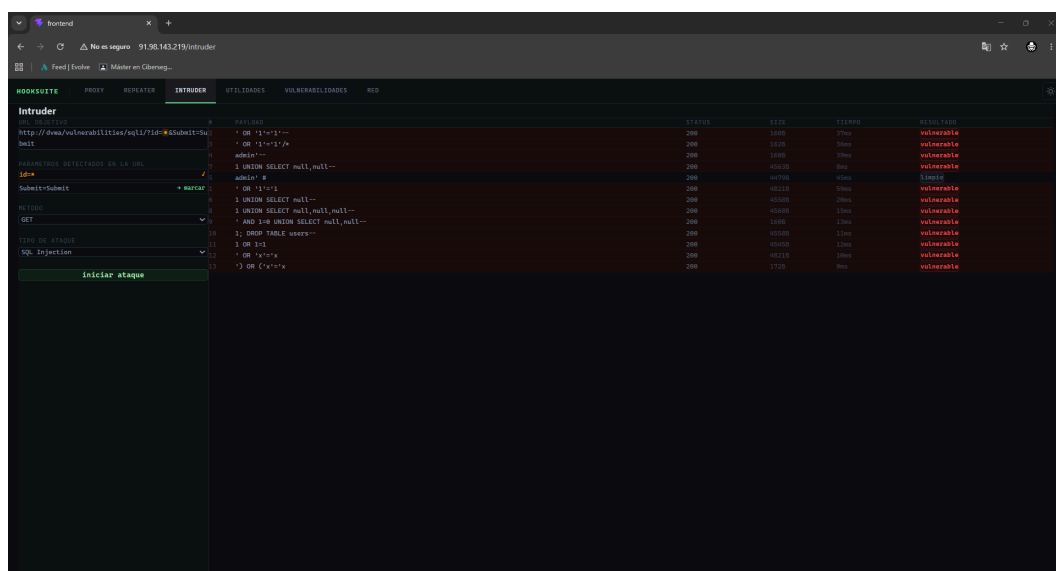


Figura 70: Resultados del Intruder — 12 de 13 payloads SQLi marcados como vulnerables en DVWA

## Utilidades

Las Utilidades agrupan cuatro herramientas auxiliares accesibles desde el topbar, útiles durante cualquier fase de la auditoría.

El **Encoder/Decoder** transforma texto entre Base64, URL encoding, HTML encoding y decodificación de JWT en tiempo real — útil para preparar payloads o analizar respuestas codificadas.

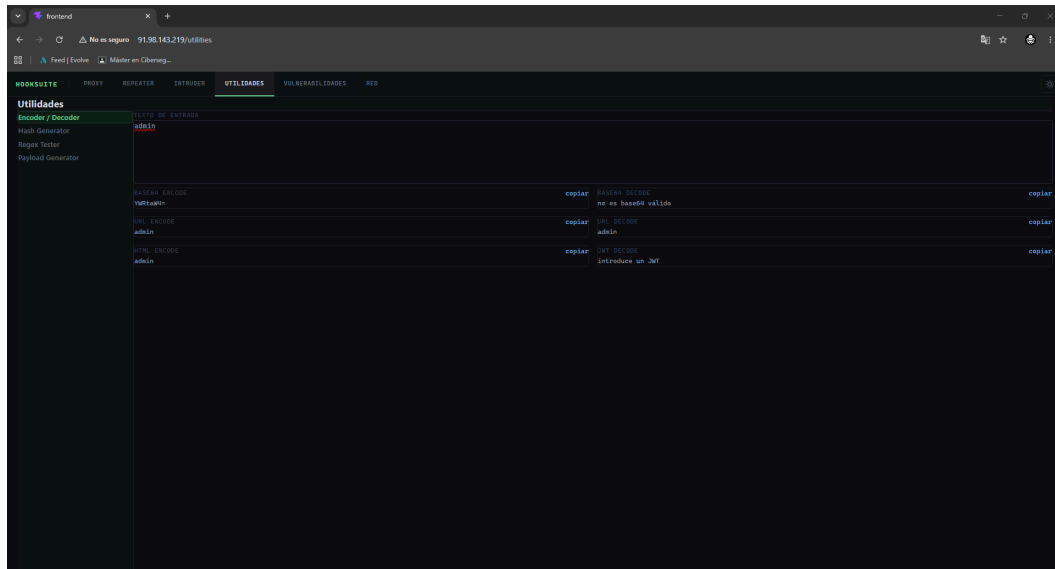


Figura 71: Encoder/Decoder — transformación de 'admin' a los distintos formatos de codificación

El **Hash Generator** calcula los hashes MD5, SHA1, SHA256 y SHA512 de cualquier texto — útil para verificar integridad de datos o preparar ataques de fuerza bruta con hashes conocidos.

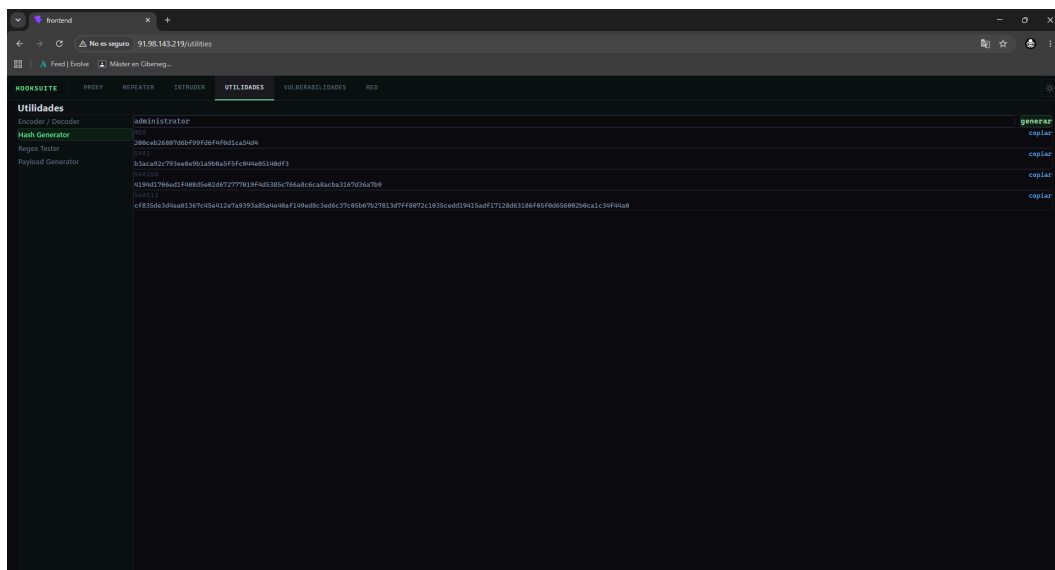


Figura 72: Hash Generator — hashes de 'administrator' en los cuatro algoritmos

El **Regex Tester** compila y ejecuta expresiones regulares contra texto de prueba con resaltado visual de los matches en tiempo real — útil para construir patrones de extracción de datos de respuestas HTTP.

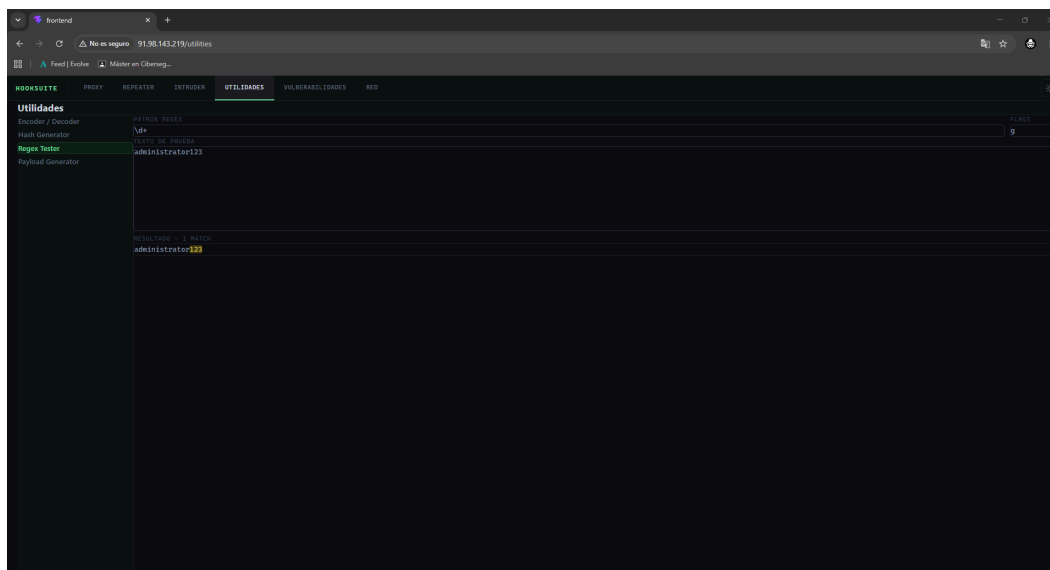


Figura 73: Regex Tester — patrón \d+ con match resaltado en 'administrator123'

El **Payload Generator** organiza colecciones de payloads por tipo de ataque con opción de copiar individualmente o todos a la vez — útil para preparar listas de payloads personalizadas antes de lanzar el Intruder.

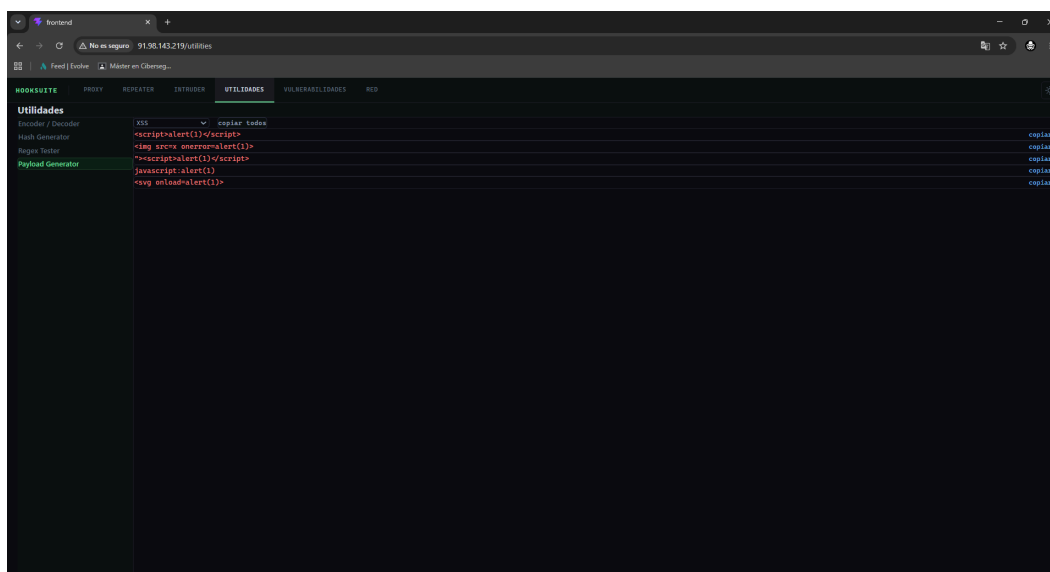


Figura 74: Payload Generator — lista de payloads XSS listos para copiar

## Paneles pendientes de activación

El panel **Vulnerabilidades** está construido e integrado en la interfaz. Está diseñado para recibir las detecciones del módulo de IA clasificadas por severidad — crítica, alta, media y baja — con descripción de la vulnerabilidad, payload utilizado y recomendación de mitigación. Su activación completa está planificada para la Práctica 2.

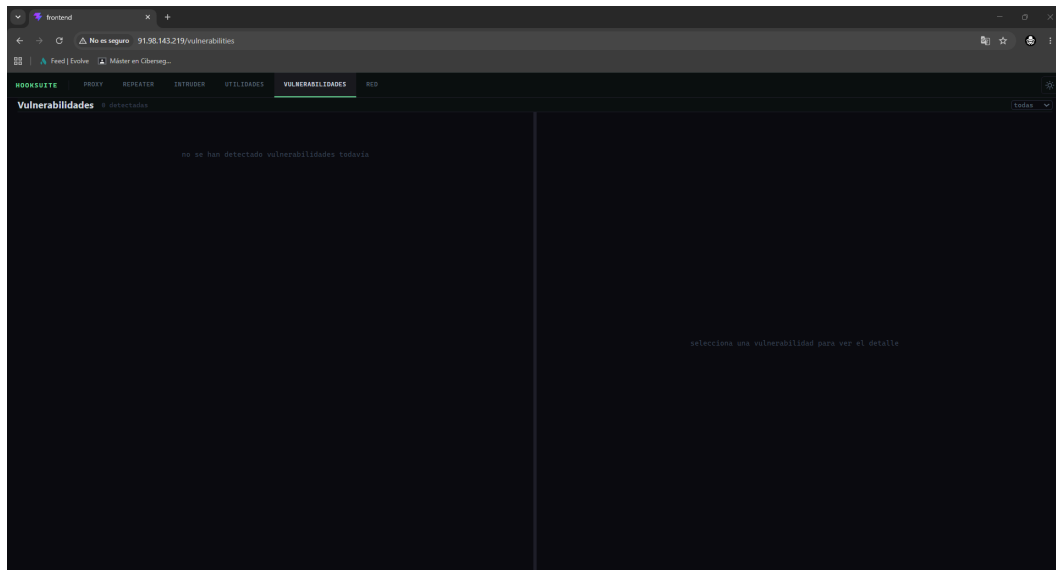


Figura 75: Panel Vulnerabilidades — construido e integrado, pendiente de activación en la Práctica 2

El panel **Red** está construido e integrado en la interfaz. Está diseñado para mostrar el tráfico capturado por el módulo DevTools en tiempo real, con código de colores para identificar peticiones limpias, sospechosas y vulnerables. Su activación completa está planificada para la Práctica 2.

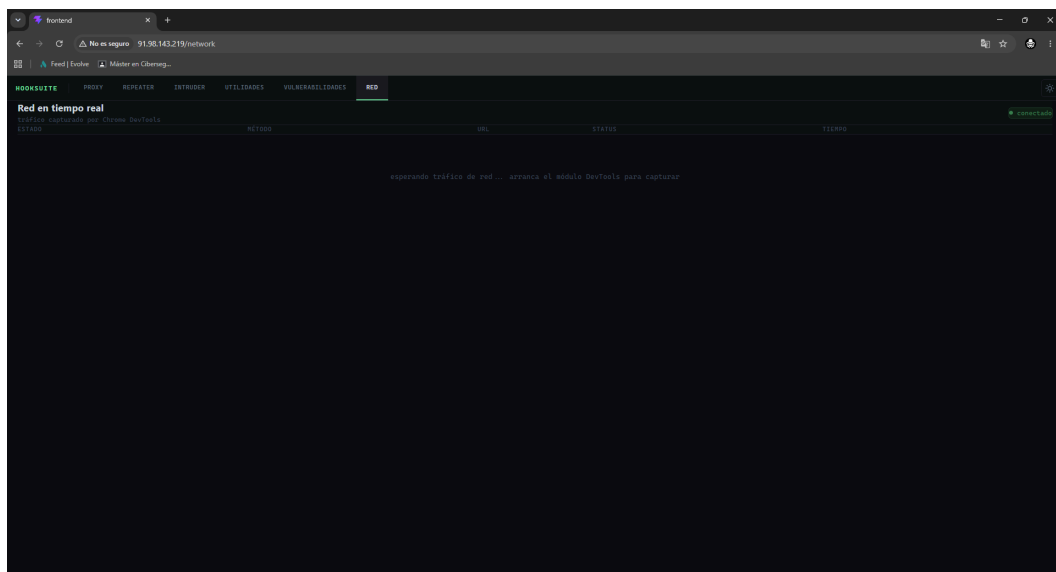


Figura 76: Panel Red — construido e integrado, pendiente de activación en la Práctica 2

## SECCIÓN 7

## Conclusiones y lecciones aprendidas

---

### Lo que funcionó mejor de lo esperado

La integración entre módulos fue más fluida de lo que el equipo anticipaba. Definir los contratos de API al inicio del proyecto —antes de que cada miembro arrancara su desarrollo— eliminó la mayoría de los problemas de compatibilidad que suelen aparecer en proyectos distribuidos de este tipo. La definición de contratos de API desde el inicio permitió que los módulos de Playwright, DevTools e IA estuvieran preparados para integrarse con el Backend sin necesidad de cambios en sus interfaces — una decisión que simplificará significativamente la integración en la Práctica 2.

El uso de variables de entorno para controlar el modo de operación del módulo de IA resultó ser una decisión especialmente acertada. El modo `mock` (`MOCK_PLAYWRIGHT=true`) permitió desarrollar y validar el ciclo completo de análisis de forma independiente, sin depender de que P2 y P3 tuvieran sus módulos listos. Esto desbloqueó el desarrollo en paralelo real y redujo los tiempos de espera entre módulos.

La elección de Claude como motor de análisis superó las expectativas iniciales en términos de precisión. La detección de inyecciones SQL en DVWA alcanzó niveles de confianza superiores al 90% con prompts relativamente sencillos, lo que confirma que la calidad del prompt es más determinante que la complejidad del código de análisis.

### Lo que fue más difícil de lo previsto

La coordinación entre cinco módulos con dependencias cruzadas generó cuellos de botella que no estaban completamente previstos en el diseño inicial. El módulo de backend actúa como nexo central del sistema, lo que convirtió a P2 en el punto de mayor presión del proyecto: cualquier endpoint pendiente en el backend bloqueaba simultáneamente a varios módulos. En retrospectiva, haber paralelizado más agresivamente el desarrollo del backend desde el inicio habría reducido estos bloqueos.

La gestión del entorno Python en Windows presentó más fricción de la esperada. La incompatibilidad entre la versión inicial de la librería `anthropic` y Python 3.14, la exposición accidental de una API key en el repositorio o la configuración del entorno virtual en distintos sistemas operativos fueron incidencias menores que, sumadas, consumieron tiempo de desarrollo que no estaba previsto en el plan inicial.

El pivote de arquitectura fue el momento más exigente del proyecto. El error de los bots al abrir el proxy TCP al exterior no estaba en ningún plan de contingencia — y no podía estarlo, porque es el tipo de problema que solo aparece cuando expones un sistema real

en internet por primera vez. La decisión de cambiar de modelo a mitad del desarrollo, con módulos ya integrados y una demo próxima, requirió que el equipo reorganizara prioridades en tiempo real y concentrara el esfuerzo donde más impactaba.

## | Qué haríamos diferente si empezáramos de nuevo

Estableceríamos la convención de commits desde el primer commit del repositorio, no como corrección posterior. La convención de commits es un estándar de calidad que penaliza directamente la nota cuando no se aplica, y su adopción tardía en un historial ya creado es costosa de corregir sin reescribir el historial.

Definiríamos un entorno de integración compartido desde el inicio. Durante el desarrollo, cada módulo se probó de forma aislada contra DVWA en local. Un entorno de integración compartido en Hetzner desde la primera semana habría permitido detectar antes los problemas de integración real entre módulos y habría dado más tiempo para resolverlos.

Diseñaríamos desde el inicio un plan de contingencia para cambios de arquitectura. El pivotar al modelo httpx fue la decisión correcta, pero llegó en un momento en que varios módulos ya estaban contruidos sobre la arquitectura anterior. Haber anticipado ese escenario — aunque fuera como un plan B documentado — habría reducido el impacto del cambio en el resto del equipo.

## | Aprendizajes sobre integración de sistemas complejos

Este proyecto confirma que la dificultad de un sistema distribuido no está en la complejidad de cada módulo individual, sino en la gestión de sus interfaces. Un módulo técnicamente excelente que no cumple el contrato de API acordado bloquea a todos los módulos que dependen de él. La disciplina en la definición y el respeto de los contratos de integración es tan importante como la calidad del código.

La inteligencia artificial como componente de un sistema mayor introduce un tipo de incertidumbre diferente al del código determinista. Los tiempos de respuesta variables y la naturaleza probabilística de las clasificaciones requieren diseñar el sistema de forma que pueda operar de forma degradada cuando la IA no está disponible o cuando su respuesta no supera el umbral de confianza requerido.

Exponer un sistema en internet real es una experiencia de aprendizaje que ningún entorno local puede replicar. El error de los bots enseñó al equipo que la seguridad no es una capa que se añade al final — es una restricción de diseño que condiciona cada decisión de arquitectura desde el principio. Un proxy TCP abierto en local es una herramienta; el mismo proxy abierto en internet es una vulnerabilidad. Esa distinción no aparece en ningún manual pero define la diferencia entre un sistema de laboratorio y un sistema real.

Trabajar en equipo distribuido con cinco módulos independientes enseña algo que el trabajo individual no puede: que la confianza técnica entre compañeros es tan importante como la competencia técnica individual. Cada miembro del equipo construyó su módulo sabiendo que otros dependían de él — y esa responsabilidad genera un nivel de cuidado en el código

que es difícil de alcanzar cuando uno trabaja solo. El pivote de arquitectura demostró además que un equipo que confía en sus decisiones colectivas puede adaptarse a cambios drásticos sin perder la cohesión.

## SECCIÓN 8

## Road map de mejora para la Práctica 2

HookSuite llega a la Práctica 2 con cinco módulos construidos pero con tres de ellos pendientes de integración completa en producción. El road map de esta fase prioriza resolver esa deuda técnica antes de añadir funcionalidades nuevas, y estructura el sistema de usuarios antes de exponer la herramienta al público general.

### Fase 1 — Integración de módulos pendientes

*Semanas 1-2 · P3, P4 y P5 construidos pero sin conectar en producción*

El objetivo de esta fase es completar el trabajo que quedó pendiente en la Práctica 1: los módulos DevTools, Playwright e IA están construidos y validados en local, pero no están integrados de forma estable en la infraestructura Docker de producción en Hetzner.

| Nº | Mejora                                   | Descripción                                                                                              | Días | Responsables |
|----|------------------------------------------|----------------------------------------------------------------------------------------------------------|------|--------------|
| 1  | Integración de DevTools (P4) en Docker   | Desplegar el contenedor y conectar el panel Red al Backend. El endpoint receptor ya está implementado.   | 3    | P4, P2       |
| 2  | Integración de Playwright (P3) en Docker | Resolver el bug de red Docker y activar el ciclo completo IA↔Playwright↔Backend en producción.           | 3    | P3, P2       |
| 3  | Activación completa del módulo IA (P5)   | Pasar de modo mock a modo polling real. El módulo arranca en el servidor — falta la integración estable. | 2    | P5, P2       |

### Fase 2 — Sistema de usuarios público

*Semanas 3-4 · Convierte HookSuite de herramienta del equipo a producto multi-usuario*

Con los módulos integrados, el siguiente paso es convertir HookSuite de herramienta de equipo a producto multi-usuario. El sistema actual usa un único usuario compartido gestionado por Nginx — esta fase lo sustituye por un sistema de registro y login individual.

| Nº | Mejora | Descripción | Días | Responsables |
|----|--------|-------------|------|--------------|
|----|--------|-------------|------|--------------|

|   |                                      |                                                                                                                    |   |        |
|---|--------------------------------------|--------------------------------------------------------------------------------------------------------------------|---|--------|
| 4 | Registro y login por usuario con JWT | Sustituye el Nginx básico. El frontend ya tiene componentes de auth preparados desde la Práctica 1.                | 3 | P2, P1 |
| 5 | Aislamiento de sesiones por usuario  | Cada usuario ve solo su historial y auditorías. El SessionManager ya gestiona tokens UUID — se vincula al usuario. | 2 | P2, P1 |

### Fase 3 — Nuevas funcionalidades

*Semanas 5-7 · Valor añadido que diferencia HookSuite de otras herramientas*

Con la integración completa y el sistema de usuarios operativo, esta fase añade funcionalidades que amplían la superficie de auditoría de HookSuite y la diferencian de otras herramientas del mercado.

| Nº | Mejora                                           | Descripción                                                                                              | Días | Responsables |
|----|--------------------------------------------------|----------------------------------------------------------------------------------------------------------|------|--------------|
| 6  | Interceptación de WebSockets                     | CDP websocketFrameReceived. Nuevo panel WS en el frontend. Las apps modernas usan WS de forma extensiva. | 5    | P4, P1       |
| 7  | Análisis JS estático con AST + endpoints ocultos | esprima/acorn para detectar URLs hardcodeadas, API keys y endpoints. Se pasan al spider automáticamente. | 5    | P4, P5, P3   |
| 8  | Integración CVEs (NVD) + exportación de informes | Cruzar fingerprint con CVEs conocidos. Exportar hallazgos en JSON y PDF auto-generado con IA.            | 5    | P5, P4, P1   |

### Fase 4 — Seguridad y producción

*Semana 8 · Con usuarios reales, el servidor necesita esto antes de la apertura pública*

Con usuarios reales accediendo a la herramienta, esta fase cierra el único aspecto de seguridad del servidor que queda pendiente. El resto de la seguridad perimetral ya está resuelta desde la Práctica 1: los puertos del servidor están cerrados por defecto, y el sistema de firewall dinámico abre un puerto exclusivo por sesión asociado al SUID del usuario, cerrándolo automáticamente al hacer logout o tras 4 horas de inactividad.

| Nº | Mejora | Descripción | Días | Responsables |
|----|--------|-------------|------|--------------|
|----|--------|-------------|------|--------------|

|   |                                             |                                                                                                                                |   |    |
|---|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---|----|
| 9 | TLS con Let's Encrypt en endpoints públicos | Configurar HTTPS en Nginx para los endpoints expuestos. El firewall dinámico por sesión ya está operativo desde la Práctica 1. | 1 | P2 |
|---|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---|----|

Tabla resumen

| Nº | Mejora                           | Fase   | Días | Responsables |
|----|----------------------------------|--------|------|--------------|
| 1  | Integración DevTools en Docker   | Fase 1 | 3    | P4, P2       |
| 2  | Integración Playwright en Docker | Fase 1 | 3    | P3, P2       |
| 3  | Activación módulo IA             | Fase 1 | 2    | P5, P2       |
| 4  | Registro y login JWT             | Fase 2 | 3    | P2, P1       |
| 5  | Aislamiento de sesiones          | Fase 2 | 2    | P2, P1       |
| 6  | Interceptación WebSockets        | Fase 3 | 5    | P4, P1       |
| 7  | Análisis JS estático + endpoints | Fase 3 | 5    | P4, P5, P3   |
| 8  | CVEs + exportación informes      | Fase 3 | 5    | P5, P4, P1   |
| 9  | TLS Let's Encrypt                | Fase 4 | 1    | P2           |

Total estimado Práctica 2: 29 días distribuidos en 8 semanas entre los 5 roles

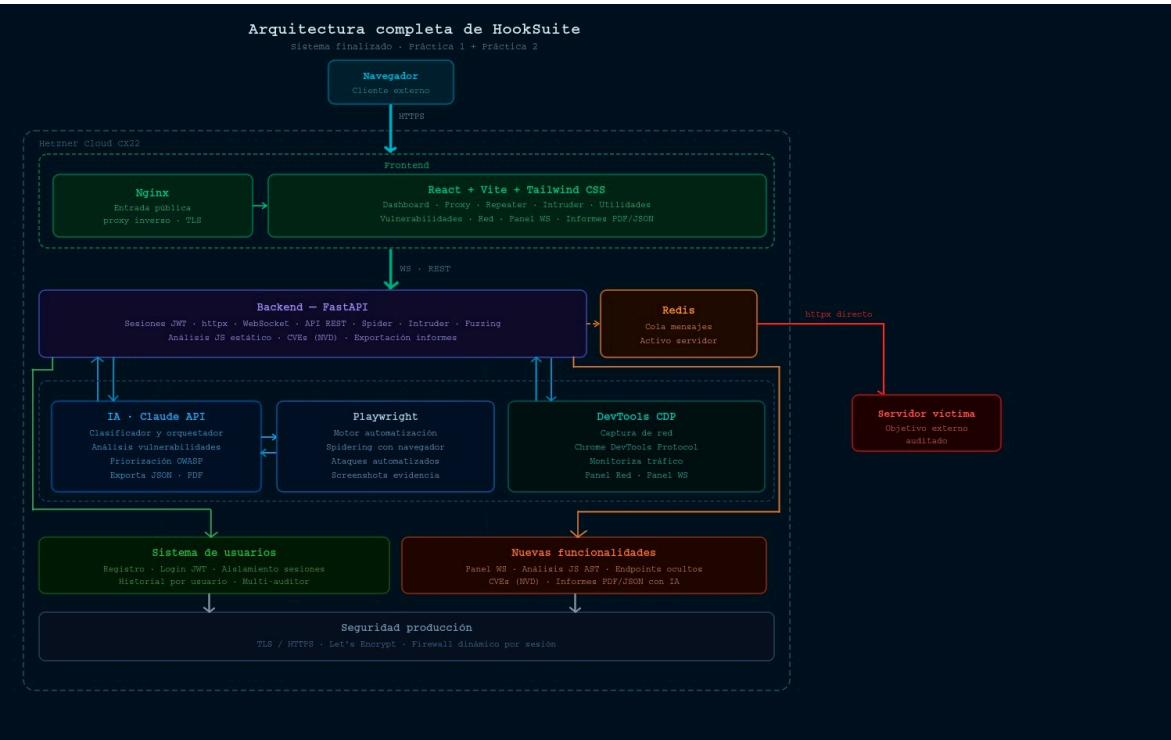


Figura 77: Arquitectura completa de HookSuite — visión objetivo Práctica 1 + Práctica 2